EspoCRM

How To Guide

Cleveland Business Mentors

Field	Value
Document	EspoCRM How To Guide
Owner	Cleveland Business Mentors
Version	2.6
Last Updated	06-12-26 20:51

Table of Contents

Entity Configuration

1. Defining an Entity

An entity in EspoCRM is a data object type — essentially a database table with a corresponding UI (list view, detail view, edit form). Custom entities are created through Entity Manager and can be tailored to fit any business need.

1.1 Overview

EspoCRM ships with built-in entities such as Contact, Account, and Meeting. When none of the built-in entities fits your use case, you create a custom entity. For CBM, the Engagement entity is an example of a custom entity created to track mentor-client assignments.

1.2 Entity Types

When you create a custom entity, EspoCRM requires you to select an Entity Type (referred to internally as a "template"). This choice determines the default fields the entity will have, which built-in side panels appear on the detail view, whether the entity supports calendar display, and how name fields are structured. The Entity Type is a structural decision made at creation time. Changing it afterward requires manual edits to PHP class definitions and JSON metadata files on the server — it cannot be done through the Admin UI. Choose carefully before saving.

Available Entity Types:

Type	What You Get	Best For
Base	Standard system fields only (Name, Description, Assigned User, Teams). No activity panels, no calendar integration.	Lookup tables and simple reference data.
Base Plus	Base features plus the Activities, History, and Tasks side panels. Records can have Meetings, Calls, Tasks, and Emails logged against them.	Most custom tracking entities (e.g., Engagement, Application).
Person	Base Plus features plus split name fields (Salutation, First Name, Middle Name, Last Name), Email, Phone, and Address. Records are flagged as containing personal data, which enables email-history matching by email address.	Any entity representing a human (Mentor, Mentee, Sponsor).
Company	Base Plus features plus organization-style Name field, Email, Phone, Billing Address, and Shipping Address.	Any entity representing an organization (Partner Company, Sponsor Organization).
Event	Base Plus features plus Date Start, Date End, Duration, Parent, and Status fields. Records appear on the EspoCRM Calendar.	Sessions, webinars, and other scheduled activities.
Category Tree	Hierarchical parent-child structure with tree navigation in the UI.	Nested category lists (e.g., topic taxonomies).

Why the Choice Matters:

EspoCRM's Admin UI does not provide a way to change the type of an existing entity. Converting an entity from one type to another after creation requires manual edits to PHP class definitions, JSON metadata files, and scope configuration on the server. In practice, the entity type should be treated as immutable once any record exists. If the wrong type is selected and discovered before records are entered, the cleanest path is to delete the entity and recreate it with the correct type.

Recommended Types for CBM Use Cases:

Use Case	Recommended Type
Person record (Mentor, Mentee, Sponsor, etc.)	Person
Organization record (Partner Company, Sponsor Organization)	Company
Scheduled session, meeting, or webinar	Event
Tracking record with activity logging (Engagement, Application)	Base Plus
Simple lookup or reference data (Industry list, Skill list)	Base
Hierarchical category list (topic taxonomy)	Category Tree

1.3 Step-by-Step: Creating a New Entity

Step 1 — Open Entity Manager

Navigate to: Admin → Entity Manager → Create Entity

Step 2 — Fill in the Entity Definition

Complete the following fields in the Create Entity form:

Field	Example	Notes
Name	Engagement	Singular, PascalCase. Used internally as the entity type identifier.
Label	Engagement	Singular display name shown in the UI.
Label (plural)	Engagements	Plural display name used in list views and navigation.
Type	Base	Determines the structural template for

		the entity (default fields, side panels, calendar integration). See Section 1.2 for a full discussion of the available types and how to choose. The type cannot be changed through the UI after creation.

Step 3 — Save

Click Save. EspoCRM creates the entity with a default set of fields (ID, Name, Created At, Modified At, Assigned User). You can then add custom fields via the Fields tab.

1.4 Adding Custom Fields

After the entity is created, navigate to: Admin → Entity Manager → [Your Entity] → Fields → Add Field

Select a field type and fill in the Name and Label. Common field types include:

- Varchar — short text (names, titles)
- Text — long-form plain text (notes, descriptions). Renders as a multi-line textarea that expands to show full content.
- Wysiwyg — rich text editor with formatting toolbar (bold, italic, lists, links, etc.). Renders formatted HTML output in the detail view, making notes and descriptions more readable. Best choice when users need to format content. Note: data is stored internally as HTML, so exported or programmatically accessed data will include HTML markup.
- Enum — dropdown with a fixed list of options
- Date / DateTime — calendar date or timestamp
- Boolean — yes/no checkbox
- Int / Float — whole number or decimal

Note: For Enum fields, if you want the field to have no default value, you must add a blank entry as the first option in the list AND leave the Default Value field empty. If the first option in the list is not blank, EspoCRM will automatically use the first item as the default value regardless of the Default Value setting. This is especially important for fields used to drive Dynamic Logic panel visibility — a blank first entry ensures no panel is shown until a value is explicitly selected.

Note: Link-type fields (relationships to other entities) are not created through the Fields tab. See Section 2 for how to create relationships between entities.

1.5 Setting an Enum Default Value Based on Another Field (Dynamic Handler)

EspoCRM does not provide a GUI option to set the default value of an Enum field based on the value of another field. This requires a custom JavaScript Dynamic Handler — a developer-level

customization that runs in the browser and responds to field value changes in real time.

Note: This approach requires custom JavaScript development and direct file access to the EspoCRM server. It is not available through the Admin UI. Only proceed if you are comfortable with JavaScript and have server file access.

Step 1 — Create the Dynamic Handler File

Create the following file on the EspoCRM server:

```
client/custom/src/my-dynamic-handler.js
```

Paste the following code into that file, replacing anotherField, targetEnumField, SpecificValue, DefaultValueA, and DefaultValueB with your actual field names and values:

```
define('custom:my-dynamic-handler', ['dynamic-handler'], (Dep) => {
    return class extends Dep {
        init() {
            this.controlFields();
            this.recordView.listenTo(this.model, 'change:anotherField', () =>
this.controlFields());
        }

        controlFields() {
            const value = this.recordView.model.get('anotherField');
            if (value === 'SpecificValue') {
                this.recordView.setFieldOptionList('targetEnumField',
['DefaultValueA']);
            } else {
                this.recordView.setFieldOptionList('targetEnumField',
['DefaultValueB']);
            }
        }
    };
});
```

Step 2 — Link the Handler to the Entity

Create or edit the entity's client metadata file on the server:

```
custom/Espo/Custom/Resources/metadata/clientDefs/YourEntity.json
```

Add the following to that JSON file, replacing YourEntity with your actual entity name:

```
{
    "dynamicHandler": "custom:my-dynamic-handler"
}
```

Step 3 — Clear Cache and Refresh

After saving both files, clear the EspoCRM cache and refresh your browser:

1. Go to: Admin → Clear Cache.
2. Hard-refresh your browser (Ctrl+Shift+R or Cmd+Shift+R).
3. Open a record of the target entity and change the controlling field value to confirm the Enum options update correctly.

Alternatively, Dynamic Logic (Admin → Entity Manager → [Entity] → Dynamic Logic) can be used to show/hide or make fields required based on another field value without custom code — but it cannot set a default Enum value. Use the Dynamic Handler approach when you need full control over which options are available based on another field.

1.6 Adding the Entity to the Left Navigation Column

After creating an entity, it is not automatically visible in the left-side navigation menu. You must add it manually through the Navbar settings.

4. Navigate to: Admin → User Interface → Navbar
5. In the Tab List section, locate your new entity in the available items on the right.
6. Drag it into the tab list on the left in the position where you want it to appear.
7. Click Save.
8. Refresh the page. The entity will now appear in the left navigation column.

Note: Each user can also customize their own navigation tabs via their personal preferences (top-right avatar → My Account → Preferences → Dashboard). The Admin-level Navbar setting controls the default layout for all users.

2. Creating Relationships

Relationships in EspoCRM define how one entity type links to another. Creating a relationship automatically generates the corresponding link field on the source entity and (optionally) a related records panel on the foreign entity.

2.1 Overview

Relationships are managed through Entity Manager, not the Fields tab. The key distinction is:

- Fields tab — for standalone data fields (text, date, dropdown, etc.)
- Relationships tab — for link fields that point to records in another entity

2.2 Relationship Types

Choose the relationship type based on how many records can exist on each side:

Type	Meaning	Example
Many-to-One	Many source records link to one foreign record	Many Engagements → one Mentor
One-to-Many	One source record links to many foreign records	One Mentor → many Engagements
Many-to-Many	Many on each side	Mentors ↔ Skills
One-to-One	One record on each side	Contact ↔ User account

2.3 What Each Relationship Type Produces

The relationship type you choose determines what gets created on each side — either a single link field (displayed in Detail View / Edit View) or a related records panel (displayed in Bottom Panels). Understanding this upfront helps you know where to go after saving a relationship to make it visible in the UI.

Type	Source Entity Gets	Foreign Entity Gets
Many-to-One	Link field (single record pointer) → configure in Detail View / Edit View	Related records panel (lists all linked source records) → configure in Bottom Panels
One-to-Many	Related records panel (lists all linked foreign records) → configure in Bottom Panels	Link field (single record pointer) → configure in Detail View / Edit View
Many-to-Many	Related records panel → configure in Bottom Panels	Related records panel → configure in Bottom Panels
One-to-One	Link field → configure in Detail View / Edit View	Link field → configure in Detail View / Edit View

The rule of thumb is:

- A link field points to one record — it appears as a single field on the record form, configured in Detail View and Edit View.
- A related records panel lists multiple records — it appears as a scrollable panel at the bottom of a record page, configured in Bottom Panels.

Note: One-to-Many is the mirror of Many-to-One — they describe the same relationship from opposite perspectives. Which type you choose when creating the relationship determines which entity is considered the source and which is the foreign entity, but the end result is identical.

2.4 Relationships Are Always Bi-Directional

When you create a relationship in EspoCRM, both sides are created simultaneously in the database. You do not need to define two separate relationships — one definition covers both directions automatically.

For example, when you create a Many-to-One relationship from Engagement to Contact, EspoCRM creates in one step:

- The Assigned Mentor link field on the Engagement entity (the source side)
- The Engagements related records panel on the Contact entity (the foreign side)

Both exist in the database from the moment you save. Whether either side is visible in the UI is a separate question — that is what the layout configuration controls (Detail View, Edit View, and Bottom Panels). But the underlying bi-directional link is always there, regardless of whether you have configured the layouts.

This explains two important things:

- The left-side fields in the Add Relationship dialog (Name and Label) are not just labels — they define the actual panel name that will appear on the foreign entity if you choose to enable it in Bottom Panels.
- Scenario A and Scenario B in Section 2.5 are independent UI choices, not separate relationship definitions. You are simply choosing which side(s) of an already bi-directional relationship to make visible.

Note: Because both sides are always created, it is worth thinking through the naming on both sides before saving — especially the left-side Label, which becomes the panel title on the foreign entity's detail page.

2.5 Step-by-Step: Adding a Relationship

Step 1 — Open Entity Manager

Navigate to: Admin → Entity Manager → [Your Entity] → Relationships → Add Relationship

Step 2 — Select Relationship Type

Choose the appropriate type from the Type dropdown. The type you select determines what gets created on each side of the relationship — a single link field or a related records panel. Refer to Section 2.3 for a full breakdown of what each type produces.

Use the following guidance to choose the right type:

- Many-to-One — use this when many records of the Source Entity (Left Side) each point to one record in the Foreign Entity (Right Side). For example, many Engagements each have one Assigned Mentor. This puts a single link field on the Source Entity

(Engagement) and a related records panel on the Foreign Entity (Contact).

- One-to-Many — use this when one record in the Source Entity (Left Side) is linked to many records in the Foreign Entity (Right Side). This is the mirror of Many-to-One — the end result is identical, but you are defining it from the opposite entity's perspective.
- Many-to-Many — use this when records on both sides can link to multiple records on the other side. For example, a Mentor can have many Skills, and a Skill can belong to many Mentors. This puts a related records panel on both sides.
- One-to-One — use this when each record on one side links to exactly one record on the other side, and vice versa. This puts a single link field on both sides.

Note: If you are unsure whether to use Many-to-One or One-to-Many, choose the entity you are currently configuring as the source and ask: can one of these records link to multiple records on the other side? If yes, use One-to-Many. If each record only ever points to one record on the other side, use Many-to-One.

Step 3 — Configure Both Sides

The dialog has a left side (source entity) and a right side (foreign entity). Fill in both:

Side	Field	Value
Left (source)	Entity	Auto-populated with the current entity
Left (source)	Name	Auto-populated (leave as default)
Left (source)	Label	Label for the related panel on the foreign entity record
Right (foreign)	Foreign Entity	The entity this field will link to
Right (foreign)	Name	Internal field name (camelCase, e.g. assignedMentor)
Right (foreign)	Label	Display label shown in the UI (e.g. Assigned Mentor)

An important point about how labels cross sides:

- The entity on the Left side of the dialog will display the Label defined on the Right side in its UI — for example, the Left entity (Engagement) will show the Right-side Label (Assigned Mentor) as a field on its detail page.
- The entity on the Right side of the dialog will display the Label defined on the Left side in its UI — for example, the Right entity (Contact) will show the Left-side Label (Engagements) as the title of its related records panel.

Note: Think of it as a cross: each side's label is what the other side shows. Before saving, confirm that both labels make sense from the perspective of the entity that will actually display them.

Step 4 — Select Filter (Optional)

The Select Filter dropdown restricts which records appear in the lookup popup when a user fills in this field. For example, filtering a Contact link to show only Mentors.

> **Note:** The Select Filter dropdown only shows filters defined at the metadata/configuration level. User-created saved searches (from the Search UI) do not appear here. If no custom filters have been configured, only 'All' will be available. Leave it set to 'All' for now; this can be refined later via metadata configuration.

Step 5 — Save

Click Save. EspoCRM will create the relationship and generate the corresponding link field on the source entity. The field will now appear in the entity's Fields list as type Link.

2.6 Making the Relationship Visible in the UI

Creating a relationship adds it to the database, but nothing will appear on screen until you configure the layouts. Where you make that configuration depends on which side of the relationship you are on and what you want to display.

A Many-to-One relationship always produces two things:

- A link field on the source entity (the "many" side) — e.g., Assigned Mentor on Engagement
- A related records panel on the foreign entity (the "one" side) — e.g., an Engagements panel on Contact

Each of these is configured in a different place. The table below explains when to use each layout type:

Layout Type	What It Controls	Example
Detail View / Edit View	A single link field on the source entity	Assigned Mentor field visible when viewing or editing an Engagement record
Bottom Panels	A panel showing all linked child records on the foreign entity	Engagements panel listing all engagements when viewing a Contact or Account record
List (Small)	Columns shown inside related records panels on other entity pages	Which Engagement fields appear in the panel summary on the Account page

Scenario A: Showing a Link Field on the Source Entity

Use this when you want to display the link field itself on a record — for example, showing which

Mentor is assigned to an Engagement when you open that Engagement.

9. Go to: Admin → Entity Manager → [Source Entity] → Layouts → Detail View
10. Drag the link field (e.g., Assigned Mentor) from the available fields panel into the layout. Click Save.
11. Repeat for Edit View so the field can be set when creating or editing a record.
12. Optionally add to List View if you want the field visible as a column in the records list.

Scenario B: Showing a Related Records Panel on the Foreign Entity

Use this when you want to display a list of all linked child records on the parent record — for example, showing all Engagements associated with an Account or Contact when you open that Account or Contact.

13. Go to: Admin → Entity Manager → [Foreign Entity] → Layouts → Bottom Panels
14. Find the related entity panel (e.g., Engagements) in the available panels list.
15. Drag it into the active area and position it where you want it to appear. Click Save.
16. To control which columns appear inside the panel, go to the child entity's List (Small) layout (e.g., Admin → Entity Manager → Engagement → Layouts → List (Small)). See Section 4 for details.

Note: Both scenarios are independent — you can configure one, both, or neither. For example, you might add the Assigned Mentor field to the Engagement Detail View (Scenario A) without adding an Engagements panel to the Contact Bottom Panels (Scenario B), or vice versa.

2.7 Example: Assigned Mentor on Engagement

The following configuration adds an Assigned Mentor link field to the Engagement entity, pointing to the Contact entity (where mentors are stored).

Side	Field	Value
Left (Engagement)	Entity	Engagements
Left (Engagement)	Name	engagements
Left (Engagement)	Label	Engagements
Right (Contact)	Foreign Entity	Contact
Right (Contact)	Name	assignedMentor
Right (Contact)	Label	Assigned Mentor

The left-side settings control how a related Engagements panel appears on a Contact record.

The right-side settings define the Assigned Mentor link field on each Engagement record.

> **Note:** With Select Filter set to 'All', the lookup popup will show all Contacts regardless of Contact Type. Users should be aware to select only Contacts marked as Mentors. A filtered lookup can be configured in a future enhancement.

2.8 Foreign Fields — Displaying Linked Entity Data

A Foreign field displays a value from a related entity by reaching through an existing link (relationship). It is a read-only mirror — the actual data lives on the related record, but it appears on this entity for display, filtering, sorting, and reporting purposes. Foreign fields are the standard way to surface data from a parent or related entity inside list views, search filters, and reports without requiring users to click through to the related record.

How Foreign Fields Work

A Foreign field requires a link (relationship) to already exist. It does not create a relationship; it follows one that has already been defined in the Relationships tab. Once configured, the Foreign field behaves like a regular field: it appears in list views, can be searched and filtered on, and can be referenced in reports and formulas. The displayed value updates when the source record changes or when the host record is linked to a different related record.

Foreign fields are most commonly used with Many-to-One links (the “many” side reaching up to the “one” side). With Many-to-Many links, behavior is undefined when the host record is linked to multiple related records, so Foreign fields should be limited to Many-to-One and One-to-One relationships in practice.

Step-by-Step: Adding a Foreign Field

The following steps assume the underlying link already exists. If the link has not been created yet, complete Section 2.5 first, then return here.

Step 1 — Open the Fields Tab

Navigate to: Admin → Entity Manager → [Entity that hosts the Foreign field] → Fields → Add Field.

Step 2 — Select the Foreign Field Type

In the field-type list, choose Foreign. The form will change to show Link and Field selectors specific to this type.

Step 3 — Choose the Link to Traverse

The Link dropdown lists the existing relationships defined on this entity. Pick the relationship that points to the entity whose data you want to display. If the link you need is not in the list, the relationship has not been created yet — return to Section 2.5 to add it.

Step 4 — Choose the Field on the Linked Entity

The Field dropdown lists the fields available on the related entity. Pick the field whose value should appear on this entity. Fields of type Varchar, Int, Float, Enum, Date, DateTime, Bool, and Text are all valid sources.

Step 5 — Set Name and Label, then Save

Enter an internal Name (camelCase, used in the API and formulas) and a Label (display name). Click Save. EspoCRM will populate the Foreign field on existing records during the next cache rebuild and on new records as they are created or linked.

Example: Mentor Email on Engagement

Building on the assignedMentor link created in Section 2.7, this configuration adds a Mentor Email Foreign field to the Engagement entity. The field will display the email address of whichever Contact is linked through assignedMentor, allowing the email to appear in Engagement list views and filters without joining to Contact in a report.

Field	Value
Type	Foreign
Name	assignedMentorEmailAddress
Label	Mentor Email
Link	assignedMentor
Field (on linked entity)	emailAddress

After saving, every Engagement record will show the linked mentor's email address as a read-only field. The field can be added to the Engagement list view, used as a search filter, and included in reports without any additional join configuration.

Important Constraints

17. Foreign fields are read-only on the host entity. To change the value, edit the source field on the related record (in the example above, edit the Contact's email address) or change which related record is linked.
18. A Foreign field requires the link to exist before the field can be created. If the link is later removed, the Foreign field will become invalid and must be removed as well.
19. When the source record changes, the displayed value updates, but workflow triggers should still be configured on the source entity rather than the host entity. Workflow conditions evaluating a Foreign field on the host entity may not always fire on source-record changes.
20. Foreign fields should be used with Many-to-One and One-to-One links. Use with Many-to-Many links is not recommended — when the host record is linked to multiple related

records, the displayed value is undefined.

2.9 One-to-One Relationships — In Depth

2.9.1 Overview

A one-to-one relationship links exactly one record on one side to exactly one record on the other side. Where many-to-one says “many engagements share one mentor” and many-to-many says “any contact can attend any session,” one-to-one says “this record corresponds to that record, and to no other on either side.”

One-to-one is the least common relationship type in practice. The CBM model uses many-to-many and many-to-one extensively but has no one-to-one relationships in production today. This is not unusual. Most data that initially looks like it calls for one-to-one is better modeled as fields directly on the parent entity (when the data is small and shares the parent’s lifecycle) or as a shared entity with a discriminator field (when several entity concepts overlap in most of their fields — Contact is CBM’s example, with contactType distinguishing Mentor, Client, Donor, and so on).

One-to-one earns its complexity when at least one of the following is true:

- The “child” data has an independent lifecycle from the parent (created, updated, or deleted on its own cadence).
- The child data is optional — most parent records have no child at all.
- The child is the CRM’s reflection of a record owned by an external system (an email platform profile, a payment gateway customer record, a third-party identity object).
- The child holds sensitive data that needs its own access control independent of the parent.
- The child is large enough that loading it on every read of the parent would be wasteful.

If none of these apply, prefer fields on the parent. If several apply, one-to-one is the right tool.

2.9.2 One-to-One Right vs One-to-One Left

EspoCRM offers two flavors of one-to-one, distinguished by which side holds the foreign key column in the underlying database table:

- One-to-One Right — the foreign key lives on the right (foreign) entity. The left (source) entity has no FK column; it reaches the right side via the link.
- One-to-One Left — the foreign key lives on the left (source) entity. The right entity has no FK column; it reaches the left side via the link.

Choosing between Right and Left is the single most consequential decision when defining a one-to-one relationship. The decision is not about which side “owns” the relationship conceptually — both sides can see and traverse the link in the UI either way. It is about where the FK column physically lives, which has three practical consequences.

Aspect	One-to-One Right	One-to-One Left
FK column location	Right (foreign) entity	Left (source) entity
Faster joins from	The right side (which carries the	The left side (which carries the

	index)	index)
Best when	The right side is queried often; most lefts have no right counterpart so a null column on the left would be wasted	The left side is queried often; most lefts do have a right counterpart
External-system mirrors	Conventional. FK on the mirror keeps the system-of-record side clean.	Less common.
Changeable after save	No (server-side metadata edits required)	No (server-side metadata edits required)

Query performance. Joins from the side that holds the FK are faster than joins from the side that does not, because the FK side carries the index. If one side is queried far more often than the other (in list views, reports, or formulas), put the FK on that side.

Optional vs mandatory shape. If most records on one side have no counterpart on the other, putting the FK on the side that is usually present keeps the side that is usually empty free of null columns. Putting the FK on the always-empty side would waste storage.

Data ownership boundaries. When the child entity represents data from an external system (an external profile, a payment customer, an identity provider record), the FK conventionally lives on the child. This makes it visible in the data model that the child is “the mirror” and the parent is the system of record on the CRM side.

When you create a one-to-one relationship through Entity Manager, the dialog presents the two options. There is no UI affordance to switch between them after the relationship is saved — converting Right to Left, or vice versa, requires manual edits to PHP class definitions and JSON metadata on the server. Decide before saving.

2.9.3 What EspoCRM Creates Under the Hood

When you save a one-to-one relationship, EspoCRM performs the following actions:

- Adds a link field on each side. The side opposite the FK gets a virtual link (resolved via reverse traversal of the FK); the side holding the FK gets a real link backed by the FK column.
- Adds the FK column to the underlying database table on the chosen side. The column is named after the link with “Id” appended (e.g., contactId, mailingPlatformProfileId).
- Adds a unique index on the FK column. This is the mechanism that enforces one-to-one at the database layer — without the unique index, the relationship would behave as many-to-one.
- Registers the link in the entity’s metadata definitions so that the API, list views, the layout manager, and the formula engine can read and write through it.

The link field appears in the Fields tab of each entity after creation, where it can be added to detail and edit layouts like any other field. The FK column itself does not appear in the Fields tab — it is managed entirely through the link.

2.9.4 Uniqueness Constraint and Cardinality Enforcement

The unique index on the FK column means that any attempt to link a target record that is already linked to another host record will be rejected at the database layer. This applies whether the link is set through the UI, the API, the formula engine, or a workflow rule. The constraint is what distinguishes one-to-one from many-to-one structurally.

The practical consequence is that to re-link a target to a different host, you must first unlink it from its current host, then link it to the new host. There is no atomic swap. The order matters: if you try to link the target to a new host before unlinking the old one, the operation fails with a unique-constraint violation and the original link remains in place.

This is the correct behavior for one-to-one, but it does surprise users who expect “link” to be a single-step operation. If your operational workflow involves frequently re-pointing the same target to different hosts, that is a strong signal that many-to-one is the correct model, not one-to-one.

2.9.5 Step-by-Step: Creating a One-to-One Relationship

Step 1 — Open Entity Manager

Navigate to: Admin — Entity Manager — [Your Entity] — Relationships — Add Relationship.

Step 2 — Select Relationship Type

In the Type dropdown, select either One-to-One Right or One-to-One Left. Refer to Section 2.9.2 for guidance on choosing. The decision cannot be reversed through the UI after saving, so make it deliberately.

Step 3 — Configure Both Sides

Configure each side’s Name and Label fields. As with all relationships in EspoCRM, both sides are configured in the same dialog — see Section 2.4 for the bi-directional behavior these fields produce on the foreign entity.

Step 4 — Save

Click Save. EspoCRM creates the link field on each side, adds the FK column with its unique index to the underlying database table, and registers the relationship in the entity metadata. Both link fields are now available to be added to layouts via the Fields tab and the Detail/Edit layout editors.

Step 5 — Add the Link Fields to Layouts

By default, neither link field appears in the detail or edit view of either entity even after the relationship is saved. To make the link visible and editable in the UI, add the appropriate link field to each entity’s Detail View and Edit View layouts (Admin — Entity Manager — [Entity] — Layouts — Detail / Edit). To show the linked record as its own panel rather than a single inline field, follow Section 3 (Adding a Related Panel to an Entity Detail View) for either side.

2.9.6 Example: Mailing Platform Profile on Contact

Consider this scenario: CBM integrates with an external email marketing platform. Each Contact who has subscribed to CBM’s email communications has exactly one profile record in the external platform; Contacts who have never subscribed have no profile at all. The profile holds

platform-side state — the external system's contact identifier, the current subscription status, the timestamp of the last sync, and the set of email lists the Contact belongs to.

This is a candidate for one-to-one because:

- The profile data has its own lifecycle (created on first subscribe, removed on hard unsubscribe, possibly re-created later).
- Most Contacts will have no profile, so the relationship is optional on the Contact side.
- The profile represents data owned by an external system, not by CBM.
- If CBM ever switches email platforms, the profile entity can be swapped out without changing the Contact entity itself.

The alternative — adding mailingPlatformContactId, mailingPlatformSubscriptionStatus, mailingPlatformLastSyncedAt, and a multi-enum mailingPlatformLists directly to Contact — also works, and is simpler. Reasonable people would choose either way. This example chooses one-to-one to illustrate the pattern; in a real implementation the platform-swappability and independent-lifecycle arguments would need to be weighed against the simplicity of fields-on-Contact.

Configuration:

Field	Value	Notes
Type	One-to-One Right	FK lives on MailingPlatformProfile (the external-system mirror); Contact has no FK column.
Left Entity (host)	Contact	The CRM-owned record. Reaches the profile via the link.
Right Entity (foreign)	MailingPlatformProfile	The external-system mirror. Holds the FK column contactId.
Left Name	mailingPlatformProfile	Singular. The link field that appears on Contact.
Left Label	Mailing Platform Profile	Display label of the link field on Contact.
Right Name	contact	Singular. The link field that appears on MailingPlatformProfile.
Right Label	Contact	Display label of the link field on MailingPlatformProfile.
Audited	Yes (both sides)	Logs link and unlink events to the activity stream of both entities.

After saving, EspoCRM creates the mailingPlatformProfile link on Contact and the contact link on MailingPlatformProfile, adds a contactId column with a unique index to the mailing_platform_profile table, and registers both link fields in entity metadata. Either link field can then be added to its entity's Detail View and Edit View layouts to make the link visible in the UI.

In use. A staff user opening a Contact detail page sees the linked MailingPlatformProfile

through the link field on Contact (or through a related panel on the Contact detail view, configured per Section 3). A staff user opening a MailingPlatformProfile detail page sees the linked Contact through the link field on MailingPlatformProfile. Attempting to link a MailingPlatformProfile that is already linked to a different Contact will fail with a unique-constraint violation; the existing link must be removed first.

2.9.7 Foreign Fields Across a One-to-One Link

Foreign fields (Section 2.8) work the same way across a one-to-one link as they do across a many-to-one link. From either side of the link, you can reach across to the linked entity and pull a field through as a read-only mirror on the host entity.

For the example above, useful Foreign fields might include:

- On Contact: mailingPlatformSubscriptionStatus, read-through from the linked MailingPlatformProfile. This allows list views and filters on Contact to show subscription status directly without joining to the profile entity in a report.
- On MailingPlatformProfile: contactEmailAddress, read-through from the linked Contact. This allows list views of profile records to show the corresponding email address without joining to Contact.

Unlike many-to-many relationships — where Foreign fields can return undefined values when the host is linked to multiple targets — one-to-one Foreign fields are always well-defined, because the linked target is always either exactly one record or none.

2.9.8 Common Pitfalls and When NOT to Use One-to-One

Re-pointing is two-step, not atomic. The unique constraint means moving a target from one host to another requires unlinking first, then linking. Workflows or formulas that “swap” the target between hosts must perform both steps in sequence. If frequent re-pointing is required by the operational workflow, many-to-one is the better fit.

Right vs Left cannot be changed through the UI. Once saved, switching the FK side requires server-side metadata edits to PHP class definitions and JSON files. Decide deliberately before saving — see Section 2.9.2.

Optional vs mandatory on each side is configured separately. A one-to-one link can be optional on both sides (the most common case), mandatory on one side, or mandatory on both. Making the link mandatory on both sides creates a “no orphans” rule that is difficult to satisfy at record-creation time, because the second-created record cannot save without the first existing first, and the first cannot save without a link to a record that does not yet exist. Mandatory-on-both is rare and usually wrong.

Reconsider before reaching for one-to-one. Most apparent one-to-one cases are better modeled as fields on the parent entity (when the data is small and shares the parent’s lifecycle) or as a shared entity with a discriminator field (when several entity concepts overlap in most of their fields, as Contact does for Mentor, Client, Donor, and so on in CBM — see the Entity Inventory). One-to-one is the right answer when the child has an independent lifecycle, optional cardinality, external-system ownership, or independent access control. If none of those apply, the simpler design is almost always better.

2.9.9 The rule for deciding one-to-OneR vs One-to-OneL

Step 1: Decide which physical entity should hold the FK (apply the conventions: optional side, or external-mirror side).

Step 2: Pick Left or Right based on where you're configuring from:

- If you're in the Entity Manager of the entity that *should* hold the FK → choose **One-to-One Left**
- If you're in the Entity Manager of the entity that should *not* hold the FK → choose **One-to-One Right**

Result is identical either way. Right and Left aren't different relationship types — they're the same relationship described from different starting points.

The 90% case

You're almost always going to be adding a 1:1 from an established entity (Contact, Account, Engagement) to a new/optional/external child entity (a profile, a mirror record, an integration log). The natural workflow is to open the **established entity's** Entity Manager and add the relationship from there.

In that scenario:

- Your established entity is on the Left (it's the one whose EM you're in)
- The new/optional/external entity is on the Right
- The FK should go on the optional/external side → that's the Right side → choose **One-to-One Right**

So the practical heuristic boils down to: **start from your main entity's EM, pick One-to-One Right.** That covers the overwhelming majority of cases.

Worked example: Contact ↔ MailingPlatformProfile

- Profile is optional (most Contacts won't have one) and mirrors an external system → FK should be on Profile.
- Natural starting point: Contact's EM (Contact already exists in your data).
- From Contact's EM: Contact = Left, Profile = Right.
- FK should be on the Right side → choose **One-to-One Right.** ✓

If instead you were defining Profile first and adding the link from Profile's EM, Profile would be Left and Contact would be Right, so you'd pick **One-to-One Left** to put the FK on Profile. Same physical relationship, same FK placement, different name in the dialog.

Quick reference

You're configuring from…	And the FK belongs on…	Pick
Established entity's EM	The other (optional/external) side	**One-to-One Right**
Optional/external entity's EM	This entity (the one you're in)	**One-to-One Left**

You're configuring from…	And the FK belongs on…	Pick
When in doubt	—	Start from the established entity's EM and use **Right**

UI Configuration

3. Adding a Related Panel to an Entity Detail View

Once a relationship exists between two entities, you can display a related records panel on the detail view of either entity. This allows users to see, add, and edit linked records without leaving the parent record's page.

A common example for CBM: after defining a relationship between Account and Engagement, you can add an Engagements panel to the Account detail view so that anyone viewing a company record can immediately see all associated engagements.

3.1 Prerequisites

Before adding the panel, confirm the following:

- A relationship between the two entities already exists (see Section 2).
- You know which entity's detail view you want to add the panel to (e.g., Account).

3.2 Step-by-Step: Adding the Panel

Step 1 — Open the Bottom Panels Layout

Navigate to: Admin → Entity Manager → Account → Layouts → Bottom Panels

Step 2 — Locate the Related Panel

The Bottom Panels layout shows all available relationship panels for this entity. Find the Engagements panel in the list. If it is currently disabled, it will appear in the disabled/hidden section.

Step 3 — Enable the Panel

Drag the Engagements panel into the active area, or click the enable toggle next to it. Reorder panels by dragging them to your preferred position.

Step 4 — Save

Click Save. Refresh the page and open any Account record — the Engagements panel will now appear on the detail view.

3.3 Using the Panel

Once the panel is visible on an Account detail page, users can:

- View all Engagements linked to that Account in a summary list.
- Click any Engagement record in the panel to open its full detail view.
- Click the Create button (plus icon) within the panel to add a new Engagement directly linked to that Account.
- Click the Edit button on any listed Engagement to modify it inline or open it for full editing.

> **Note:** The columns shown in the panel are controlled by the Engagement entity's List (Small) layout, not the Account layout or the regular List View. To change which fields appear in the panel summary, go to: Admin → Entity Manager → Engagement → Layouts → List (Small). See Section 4 for details.

4. Configuring Fields Shown in a Relationship Panel

When a related records panel appears on an entity detail page (e.g., the Engagements panel on an Account), it displays a summary list of linked records. The columns shown in that summary are controlled by a separate layout called List (Small) — not the regular List View and not the parent entity's layout.

4.1 Understanding the Layout Types

EspoCRM uses two distinct list layouts for each entity:

- List View — controls the columns shown on the entity's own main list page (e.g., when you click Engagements in the left navigation and see all engagements).
- List (Small) — controls the columns shown when the entity appears as a related records panel inside another entity's detail page (e.g., the Engagements panel on an Account or Contact record).

These two layouts are independent. Changing one does not affect the other. For relationship panels, you always edit List (Small).

4.2 Step-by-Step: Configuring the Panel Columns

Step 1 — Open the List (Small) Layout

Navigate to: Admin → Entity Manager → [Child Entity] → Layouts → List (Small)

For example, to configure the columns shown in the Engagements panel on an Account page, go to: Admin → Entity Manager → Engagement → Layouts → List (Small)

Step 2 — Review the Current Columns

The layout editor shows the fields currently included as columns on the left, and available fields that are not yet included on the right. EspoCRM typically pre-populates List (Small) with just the Name field.

Step 3 — Add Fields

Drag fields from the available list on the right into the active columns on the left. Position them in the order you want them to appear left to right in the panel.

For an Engagements panel, useful fields to consider adding include:

- Assigned Mentor — shows which mentor is linked to the engagement
- Status — shows the current state of the engagement
- Start Date — when the engagement began
- Description — a brief summary of the engagement

Step 4 — Save

Click Save. Refresh the page and open a parent record (e.g., an Account) — the panel will now display the updated columns.

4.3 Column Width Considerations

Related panels have limited horizontal space compared to the full list page. If you add too many columns, they will either wrap, truncate, or cause horizontal scrolling. As a general guideline:

- Keep List (Small) to 3–4 columns for comfortable readability.
- Prioritize the most important at-a-glance fields — status, key date, and the most relevant link field.
- The Name field is usually kept as the first column since it is also the clickable link to open the full record.

Note: List (Small) changes affect every panel where that entity appears as a child — not just

one specific parent entity. For example, if you update the Engagement List (Small) layout, the Engagements panel will show the same columns on both Account and Contact detail pages.

5. Controlling Panel Visibility with Dynamic Logic

Dynamic Logic allows panels and fields on an entity's detail view to be shown or hidden based on the value of another field on the same record. This is useful when different record types require different layouts — for example, showing a Mentor Details panel only when a Contact's type is set to Mentor, and a Client Details panel only when the type is set to Client.

5.1 How It Works

Dynamic Logic conditions are evaluated in real time as a record is viewed or edited. When a field value changes, the visibility of any panels or fields that depend on that condition updates immediately without requiring a page refresh.

Conditions can be based on any field on the entity, including:

- Enum (dropdown) fields — show a panel when the dropdown equals a specific value
- Boolean fields — show a panel when a checkbox is checked
- Link fields — show a panel when a relationship field is or is not empty

Note: When using an Enum field to drive Dynamic Logic, ensure the first option in the Enum list is a blank entry and the Default Value is also left blank. If the first option is not blank, EspoCRM will assign it as the default, which will cause a panel to be shown immediately on new records before a user has intentionally selected a value. See Section 1.4 for details on configuring Enum fields correctly.

5.2 Step-by-Step: Configuring Dynamic Logic for a Panel

Step 1 — Open Dynamic Logic

Navigate to: Admin → Entity Manager → [Your Entity] → Dynamic Logic

Step 2 — Locate the Panel

The Dynamic Logic editor lists all panels available on the entity. Find the panel whose visibility you want to control and click on it to open its condition editor.

Step 3 — Set the Visible Condition

In the Visible condition section, define the rule that must be true for the panel to be displayed. Click Add Condition and configure:

- Field — select the field whose value will drive the visibility (e.g., Contact Type).
- Operator — select the comparison operator (e.g., Equals, Not Equals, Is Empty, etc.).
- Value — enter or select the value to compare against (e.g., Mentor).

Multiple conditions can be combined using AND or OR logic if the visibility depends on more than one field value.

Step 4 — Save

Click Save. The condition takes effect immediately. Open a record of that entity type and change the controlling field value to confirm the panel shows and hides correctly.

5.3 Example: Contact Type Driving Panel Visibility

The CBM Contact entity uses Contact Type (an enum field with values Mentor and Client) to control which detail panels are displayed. The configuration is:

Panel	Visible When	Condition
Mentor Details	Contact Type = Mentor	Contact Type Equals Mentor
Client Details	Contact Type = Client	Contact Type Equals Client

When a Contact record is opened, only the panel matching the current Contact Type value is displayed. If Contact Type is not yet set, neither panel is shown until a value is selected.

Note: Dynamic Logic conditions apply to the detail view and edit view only. They do not filter which records appear in list views or relationship panels — for that, use search filters or saved views.

6. Organizing Fields into Tabs on the Detail View

For entities with a large number of fields, EspoCRM supports organizing panels into tabs on the detail view. This reduces scrolling and improves usability by grouping related fields under clickable tab headers. Tab support is available as of EspoCRM v7.2.

6.1 How Tabs Work

Tabs are created by designating certain panels as Tab-Break panels in the Detail View layout. A Tab-Break panel marks the start of a new tab — all panels following it (until the next Tab-Break) are grouped into that tab. The Tab-Break panel's label becomes the tab header displayed at the top of the detail view.

6.2 Step-by-Step: Adding Tabs to a Detail View

Step 1 — Open the Detail View Layout

Navigate to: Admin → Entity Manager → [Your Entity] → Layouts → Detail View

Step 2 — Edit a Panel to Make It a Tab-Break

Click the pencil (edit) icon on the panel you want to be the first panel in a new tab. In the panel settings:

- Check the Tab-break option to designate this panel as the start of a new tab.
- Enter a Tab Label — this is the text that will appear on the tab header (e.g., Personal Info, Training, Engagement Details).

Step 3 — Organize Additional Panels

All panels below a Tab-Break panel are automatically grouped into that tab, until the next Tab-Break panel is encountered. To create additional tabs, repeat Step 2 for each panel that should start a new tab.

For example, if you have panels A, B, C, D, E and you mark A and C as Tab-Break panels:

- Tab 1 (label from A) — contains panels A and B
- Tab 2 (label from C) — contains panels C, D, and E

Step 4 — Save and Clear Cache

21. Click Save in the layout editor.
22. Go to: Admin → Clear Cache.
23. Refresh your browser. The detail view will now show tabs at the top.

6.3 Important Notes

- Tab-Break panels should not be combined with the Hidden panel parameter — they are not fully compatible.
- Dynamic Logic conditions can be used to control the visibility of both individual panels and entire tabs based on field values, the same way as documented in Section 5.
- If only one tab is defined, EspoCRM will display the layout without tabs. At least two Tab-Break panels are needed for the tab interface to appear.

Note: Tab organization only affects the Detail View and Edit View. It has no effect on List View, Bottom Panels, or List (Small) layouts.

Email Configuration

EspoCRM includes a full-featured email system that serves as a central hub where emails from multiple accounts are gathered and linked to CRM records. It is not a personal email client replacement, but rather a shared communications layer that connects email activity to contacts, accounts, engagements, and other records.

5. Understanding How EspoCRM Handles Email

5.1 Core Concept

Unlike a personal email client, EspoCRM is designed so that the same email record can appear in the inboxes of multiple users — for example, when multiple users were recipients of the same message. When the same email is fetched from multiple email accounts, only one email record is stored in EspoCRM, identified by its Message-ID to prevent duplicates.

Emails are fetched automatically by the system in the background every few minutes. The fetch interval can be configured by the administrator.

A user with sufficient permissions can see emails that were not addressed directly to them, which allows administrators to monitor team communications where needed.

5.2 Email Status Values

Each email record in EspoCRM has a Status field with one of the following values:

- Sent — the email was composed and sent from within EspoCRM.
- Imported — the email was fetched from an IMAP account or imported manually.
- Draft — the email was composed but not yet sent.

5.3 How Emails Are Linked to Records

When a new email arrives, EspoCRM attempts to automatically recognize which record it belongs to and link it accordingly. It can link to Accounts, Contacts, Leads, Opportunities, Cases, and other entities based on matching email addresses.

If the system cannot automatically link an email, the user can link it manually by filling in the Parent field on the email record.

All emails linked to a specific record are shown in the History panel of that record. If an email is

linked to a related record (for example, an email linked to an Engagement that is itself linked to an Account), the email will appear in the History panel of both records.

Note: If an email comes from a new potential contact, the user can convert it to a Lead from the top-right menu of the email record. It is also possible to create a Task or Case directly from an email record using the same menu.

6. The Emails List View

6.1 Default Folders

The Emails list view is accessed via the Emails tab in the navigation. The following default folders are available to all users:

- All — all emails in the system the user has access to, including emails belonging to other users if Roles permit.
- Inbox — incoming emails addressed to the user are automatically placed here.
- Sent — emails sent by the user from within EspoCRM.
- Draft — emails composed but not yet sent.
- Archive — inbox emails that the user has explicitly moved to the Archive.
- Trash — inbox emails that the user has explicitly moved to the Trash.

In addition to the default folders, personal Email Folders and Group Email Folders created by the user or administrator will also appear in the list.

6.2 Inbox Behavior

Only emails in the Inbox can be:

- Moved to Archive, Trash, or a personal Email Folder by the user.
- Marked as read, unread, or important by the user.

The same email record can appear in the Inbox of multiple users simultaneously — for example, if multiple users were recipients of the same message.

Note: When removing an email it is removed from the system entirely and will disappear from all inboxes. To avoid permanent deletion, use Move to Trash instead. Administrators can restrict delete access for users to prevent accidental email loss.

6.3 Dragging Emails to Folders

On the list view, emails can be moved between folders by dragging them by their subject link

into the desired folder. This feature is available as of EspoCRM v7.3.

7. Sending Emails

7.1 Ways to Compose an Email

A new email can be composed in several ways:

- Using the Compose Email button on the Emails list view.
- Replying to an existing email.
- Clicking on an email address displayed on any record (such as a Contact or Account).
- Using the Compose Email action from the Activities panel on a record's detail view.

7.2 Email Templates

When composing an email, users can select a pre-defined template to populate the subject and body. Templates are managed separately and can be set up by an administrator for common communication types such as welcome messages, follow-ups, or status updates.

7.3 Email Signature

Each user can configure a personal email signature that will be automatically appended to all emails they compose. To set up a signature, go to: top-right avatar → Preferences → Email Signature.

7.4 Scheduled Send

When composing an email, it can be scheduled to be sent at a specific future time rather than immediately. The email is saved to Drafts and the scheduled time can be changed after saving. To schedule a send, use the dropdown menu next to the Send button when composing.

7.5 Using an External Email Client

If a user prefers to use their device's default email client instead of composing within EspoCRM, this can be enabled in their preferences. Go to: top-right avatar → Preferences → check Use an External Email Client.

8. Email Folders

8.1 Personal Email Folders

Users can create their own personal Email Folders to organize emails. To create or manage personal folders, go to: Emails list view → dropdown in the top-right corner → Folders.

When creating a folder, the Skip Notification option can be enabled to suppress notifications for emails that are automatically routed to that folder.

Note: Email folders in EspoCRM are not connected to IMAP folders. There is no two-way sync between EspoCRM folders and folders in an external email client or mail server.

8.2 Group Email Folders

Group Email Folders are shared folders accessible by teams. They are created by an administrator, who specifies which teams have access to each folder. Group folders appear alongside personal folders in the Emails list view for users who have access.

Key behaviors for Group Email Folders:

- Inbound emails from a group email account can be automatically routed to a specific group folder.
- Emails matching an email filter can be automatically moved to a group folder.
- If an email is moved from a group folder to a personal folder or the Inbox, it is unlinked from the group folder.
- Users can only move emails to a group folder if they have edit access to that email.
- Field-level security in Roles can be used to restrict the ability to change the group folder field on an email.

Note: Group Email Folders are available as of EspoCRM v7.3.

9. Email Filters

Email filters allow emails to be automatically processed based on criteria such as sender, subject, or other fields. There are two types of email filters:

- Skip — the email is placed in Trash or not imported at all if the filter is associated with a Personal Email Account.
- Put in Folder — imported emails matching the filter criteria are automatically placed in a specified folder.

9.1 Global Email Filters

Administrators can create global email filters that apply system-wide to skip undesirable emails. These are managed at: Admin → Email Filters.

9.2 Personal Email Filters

Regular users can create their own email filters that apply to their Personal Email Accounts or their entire inbox. These are managed at: Emails → dropdown in the top-right corner → Filters.

10. Message Templates and the Template Manager

EspoCRM generates a number of system emails automatically — meeting and call invitations, assignment and mention notifications, and password-reset and access-info messages, among others. The wording of these messages is controlled from a single place: Admin → Template Manager. This is a different feature from the Email Templates used in workflows and manual sends (Section 7), and from PDF Templates; the three are easily confused and behave differently.

10.1 What the Template Manager Controls

The Template Manager holds a fixed list of system message templates. Any template in the list can be edited, but new entries cannot be added through the UI — the set is defined by the system. For CBM the templates that matter most are the meeting and call invitations sent to mentors and clients, and the notification a staff member receives when an Engagement or other record is assigned to them.

10.2 Editing a Template

Open Admin → Template Manager and select the template to change (for example, Invitation: Meeting). Edit the Subject and Body, then Save. To insert a record's field value into the text, write the field name in double curly braces — for example {{dateStart}} or {{status}}. The name in the braces is the system field name shown at Admin → Entity Manager → [entity] → Fields, not the field's display label.

10.3 Template Manager vs. Email Templates

These are not the same feature and do not share the same capabilities. Email Templates (Section 7) drive workflow and manual sends, provide a Placeholders dropdown, and have broad access to record and related data. Template Manager templates are system-level and more restricted. A placeholder that resolves correctly in an Email Template — particularly one reaching a related record's field — can render as blank in a Template Manager message with no error raised. When a system email arrives with a value missing, this mismatch is the first thing to check.

10.4 Reaching Related-Entity Data

By default a system template can only read fields on the record itself, not fields on related entities, and for invitations this restriction is enforced deliberately for performance. A cross-entity placeholder will simply not resolve. Editing the core PHP to relax this works but is overwritten on the next upgrade. The upgrade-safe approach is to add a Foreign field (Section 2.8) on the entity so the related value is mirrored locally, then reference that field in the template. For

example, to show the assigned mentor's email address in a message about an Engagement, add a Foreign field that pulls email across the assignedMentor link and reference that field, rather than attempting to reach Contact directly from the template.

Template helpers behave the same way in system templates as they do in PDF Templates, so any custom helper created for one is available in the other.

Lists and Filter Configuration

This section will document how to create and manage filtered list views in EspoCRM, including system-level saved searches available to all users and views that dynamically filter by the current logged-in user.

Topics planned for this section — pending research:

- 5. Creating a My Engagements view — a filtered list showing only engagements where the current user is the Assigned Mentor
- 6. Creating system-level saved searches — making a saved filter available to all users, not just the creator

Note: See the Questions for Research section for the open questions that need to be resolved before these topics can be fully documented.

7. Reports and Report-Based Filters

EspoCRM Reports are an Advanced Pack feature that produces lists or grids (counts, sums, averages) of records matching a defined filter. Reports can be run interactively, mounted as dashlets, or attached to a record detail view as a related panel. They can also feed list-view 'Report Filters', a separate feature that exposes a report's filter logic as a primary filter on the entity's normal list view.

This section covers all three filter concepts in the Reports model, working from the most general (record filters in the report itself) to the most specific (the list-view Report Filters admin feature). The term "filter" is overloaded across these three concepts — the terminology table in 7.1 disambiguates them upfront.

7.1 Terminology — Three Filter Concepts

EspoCRM uses the word "filter" for three different things in the Reports model. Operators commonly conflate them. The three concepts, the vocabulary EspoCRM actually uses, and where each lives in the admin:

Concept	EspoCRM name	Where configured	Purpose
Record filter	"Filters" (inside the report)	Reports — [Report] — Filters tab	Conditions that determine which records appear in the report results. SQL WHERE-clause equivalent.

Runtime filter	“Runtime Filters” (inside the report)	Reports — [Report] — Runtime Filters tab	Filter slots defined in the report but supplied with values when the report is run, mounted, or attached to a record.
List-view filter from a report	“Report Filters” (administration feature)	Administration — Report Filters	Reuses a report’s filter logic as a primary filter on the entity’s normal list view, dashlets, and certain formula functions.

Throughout the rest of this section, the term “record filter” refers to the first concept, “runtime filter” to the second, and the capitalised term “Report Filter” (singular — matching the EspoCRM admin label) refers specifically to the third.

7.2 Record Filters — The Seven Constructs

Record filters are conditions baked into the report definition that constrain which records appear in results. Configured on the Filters tab of the report. Equivalent to a SQL WHERE clause. EspoCRM offers seven distinct filter constructs, combinable via grouping:

- **Field filter —** filters by specific fields of the target entity or fields of related records. The everyday case. Example: filter Engagements by their Account’s industry.
- **OR group —** composite filter satisfied if at least one inner condition is met.
- **AND group —** composite filter satisfied only if all inner conditions are met.
- **NOT IN group —** exclusion via sub-query. Avoid when an equivalent ‘Not Equals’ or ‘None of’ field filter would work — the sub-query can affect performance on large data sets. For CBM volumes this is rarely a concern, but the habit is still worth keeping.
- **IN group —** AND-like sub-query. Use when records must match conditions across multiple related rows simultaneously — for example, Accounts that have Engagements in both ‘Active’ and ‘On-Hold’ status. Plain AND groups cannot express this because each related Engagement evaluates the conditions independently.
- **Complex Expression —** applies a database function or expression to a column and compares the result to a formula expression. The most powerful and least intuitive construct. Covered in depth in 7.4.
- **Having group —** filters on aggregate values (COUNT, SUM, MAX, MIN, AVG). Only meaningful in Grid reports. Example: Accounts with more than one Engagement.

7.3 Runtime Filters — Three Usage Modes

Runtime filters are defined in the report but their values are supplied at run time, not stored in the report. The same runtime filter slot behaves differently depending on how the report is being used. There are three modes:

- **Interactive run —** when a user runs the report directly, EspoCRM prompts for the runtime filter values before executing. Useful for date-range pickers, status selectors, or any input that should change per execution.
- **Dashlet filter override —** when the report is mounted as a dashboard dashlet, runtime filters become per-dashlet settings. Different users (or the same user on different dashboards) can apply different values without modifying the underlying report. Two users’ “My Engagements (Active)“ dashlets can each scope to their own User without

two separate reports.

- **Report panel auto-fill —** when the report is mounted as a related panel on a record's detail view, a runtime filter of type Link, Link-Multiple, or Link-Parent is automatically populated with the parent record. For example, a Sessions report with a runtime Engagement filter, mounted as a panel on the Engagement detail view, will automatically scope itself to the Engagement being viewed. This is the mechanism by which report panels "follow" the current record.

Limitation: Runtime filters are not supported in Joint reports. If a Joint report is needed, all filter conditions must be expressed as record filters in each of the joined sub-reports.

7.4 Complex Expression — In Depth

A Complex Expression is a small expression language that EspoCRM translates into a SQL fragment. In a report filter, it occupies the left side of the comparison; the right side is either a literal value or a Formula expression (a separate, different syntax). The middle is a comparison operator (Equals, Greater Than, etc.).

The two sides are evaluated at completely different times and have different syntaxes — this is the single most confusing aspect of the feature. Complex Expression uses **UPPER_CASE: (args)**; Formula uses **lowercase(args)** with backslashes for namespaces (e.g., **datetime\ today()**).

7.4.1 Syntax Rules

- **Function names are UPPER_CASE** with a mandatory trailing colon, followed immediately by parentheses with no whitespace between. Example: **CONCAT: (firstName, ' ', lastName)**. Whitespace inside the parens is fine; whitespace between the colon and the open-paren is not.
- **Attribute names are lowerCamelCase** and match the field names visible in Entity Manager: **firstName**, **dateStart**, **assignedUserId**.
- **Related entity attributes use dot notation.** Examples: **account.industry**, **assignedUser.name**, **parent.createdAt**. The "from" side of the relationship is the report's target entity type.
- **String literals require single quotes.** Example: **'Completed'**, never double-quoted.
- **Functions nest freely.** A function call can be any argument: **LESS_THAN: (TIMESTAMPDIFF_MONTH:(createdAt, NOW:()), 12)**.
- **Special attribute name patterns** for non-scalar field types:
- Link field named **account** → use **accountId** (the foreign key) or **accountName** (denormalized name cache), not **account**.
- Link-Parent field named **parent** → use **parentId** and **parentType**.
- Currency field named **amount** → use **amount** (numeric value) and **amountCurrency** (ISO code).

7.4.2 The Formula Side — What Works, What Doesn't

Per the EspoCRM documentation, formula functions intended to interact with the entity record will not work inside a Complex Expression filter, because the formula is not applied to a specific record. The formula runs once before the SQL executes, not per-row.

Works on the right side of the comparison:

- Time-of-evaluation functions: **datetime\today()**, **datetime\now()**.

- Derived time values: **datetime\addMonths(datetime\today(), -3)**.
- Current-user context: **env\userAttribute('id')**.
- Pure arithmetic, string utilities, and any function that does not reference a specific record.

Does not work on the right side:

- **$variable** references.
- **entity\attribute(...)** referring to the current row's fields.
- Anything that implies "this record."

If a filter must compare two columns on the same row, both sides must be encapsulated in the Complex Expression itself, which returns 0 or 1 (false or true) and is then compared to the literal **1**. See Example 3 in 7.4.4 for the pattern.

7.4.3 Function Reference

Grouped by purpose. The full list is in EspoCRM's Complex Expressions documentation; this is the operator's working subset.

Conditional logic

- **IF:(condition, valueIfTrue, valueIfFalse)** — two-branch.
- **SWITCH:(cond1, val1, cond2, val2, ..., default)** — multi-branch (SQL CASE WHEN). v7.4+.
- **MAP:(key, k1, v1, k2, v2, ..., default)** — value lookup (SQL CASE). v7.4+. Most common use is collapsing an enum into a coarser grouping — see Example 4 in 7.4.4.

Comparison (returns 0 or 1)

- **EQUAL**, **NOT_EQUAL**, **GREATER_THAN**, **LESS_THAN**, **GREATER_THAN_OR_EQUAL**, **LESS_THAN_OR_EQUAL**.
- **LIKE:(field, 'pattern%')**, **NOT_LIKE** — SQL LIKE pattern matching with **%** wildcard.
- **IS_NULL**, **IS_NOT_NULL** — unset checks. Use these instead of **EQUAL:(field, NULL)**, which never matches.
- **IN:(field, 'v1', 'v2', ...)**, **NOT_IN** — value-set membership.
- **COALESCE**, **IFNULL**, **NULLIF** — NULL handling.
- **GREATEST**, **LEAST** — max / min across arguments. v7.4+.

Date and time

- **Component extraction: YEAR**, **MONTH_NUMBER** (1-12), **MONTH** (e.g. **'2026-05'**), **QUARTER** (e.g. **'2026_1'**), **WEEK_NUMBER_0** / **WEEK_NUMBER_1** (Sun vs Mon start), **DAYOFWEEK** (1-7), **DAYOFMONTH** (1-31), **HOUR** (0-23), **MINUTE** (0-59), **DATE** (date part of a datetime).
- **Fiscal-year variants: YEAR_X** and **QUARTER_X** where X is the month number (0-11) the fiscal year starts in. **YEAR_3** = fiscal year starting in April.
- **Current time: NOW:()** (no args; trailing colon is still required).
- **Time-zone offset: TZ:(timestampField, offsetHours)**, e.g. **TZ:(createdAt, -5)** for US Eastern.
- **Difference between datetimes: TIMESTAMPDIFF_YEAR**, **_MONTH**, **_WEEK**, **_DAY**, **_HOUR**, **_MINUTE**, **_SECOND**. Result is in the named unit.

- **UNIX_TIMESTAMP** — returns Unix epoch seconds.

String

- **CONCAT**, **LEFT:(s, n)**, **LOWER**, **UPPER**, **TRIM**, **CHAR_LENGTH**, **REPLACE:(haystack, needle, replacement)**.
- **BINARY** — forces case-sensitive comparison. MySQL collations are case-insensitive by default, so **EQUAL:('test', 'Test')** returns true. Wrap with **BINARY** if case must be respected.

Math

- **ADD**, **SUB**, **MUL**, **DIV**, **MOD**, **FLOOR**, **CEIL**, **ROUND:(value, decimals)**.

Logical

- **OR:(a, b, c)**, **AND:(a, b, c)**, **NOT:(a)**. These compose Complex Expression sub-expressions. The OR / AND / NOT groups in the report Filters UI are usually easier; reach for these only when the nesting goes beyond what the UI supports.

Miscellaneous

- **ANY_VALUE** — picks an arbitrary value from an aggregated group. v9.1.6+. Useful in Grid reports with non-aggregated columns.

7.4.4 Worked Examples for CBM-Shaped Problems

Each example shows a real CBM business question, the Complex Expression to enter in the report's Filters tab, the comparison operator, and the right-side value. Field names are taken from the Engagement and Session Entity PRDs.

Example 1 — Engagements with a session gap exceeding 1.5x their meeting cadence.

Activity Monitoring (MN-INACTIVE) flags engagements where the gap since the last session exceeds the expected cadence by a meaningful margin. Field filters cannot express this because the threshold depends on another column on the same row (**meetingCadence**).

```
SUB:(
    TIMESTAMPDIFF_DAY:(lastSessionDate, NOW:()),
    MUL:(
        MAP:(meetingCadence,
            'Weekly',     7,
            'Bi-Weekly',  14,
            'Monthly',    30,
            'As Needed',  60
        ),
        1.5
    )
)
```

Operator: Greater Than **Value:** 0

Reads as: (days since last session) minus (expected gap days x 1.5). A positive result means the engagement is more than 1.5x past its expected next-session date. Three constructs in one expression: **TIMESTAMPDIFF_DAY** for the actual gap, **MAP** to translate the cadence enum to a numeric expected-gap, and **MUL** for the tolerance factor.

Example 2 — Engagements still On-Hold past their planned hold end date.

holdEndDate is the date the mentor expects to resume an On-Hold engagement. When today's date passes **holdEndDate** and the status is still On-Hold, someone needs to act.

```
TIMESTAMPDIFF_DAY:(holdEndDate, NOW:())
```

Operator: Greater Than **Value:** 0

Pair this with a separate field filter **engagementStatus = On-Hold** in the same report. The Complex Expression handles the date math; the field filter handles the status check. Cleaner than encapsulating both in a single boolean Complex Expression.

Example 3 — Sessions logged before their parent Engagement was created (data quality check).

No Session should have a **dateStart** earlier than its parent Engagement's **createdAt**. Records that violate this almost certainly came from a bad migration or data-entry mistake. This is the textbook cross-entity column comparison.

```
LESS_THAN:(dateStart, parent.createdAt)
```

Operator: Equals **Value:** 1

Two things to call out. First, **parent.createdAt** uses the native Event **parent** link (per Session Entity PRD SES-DEC-008) to reach the Engagement. Second, the expression returns 1 (true) or 0 (false), and we filter for 1. This is the standard pattern when the entire comparison must happen on the row side, since Formula cannot reference row attributes.

Example 4 — Grid report grouping by engagement lifecycle phase (Group By, not Filter).

A dashboard chart should group engagements into four lifecycle phases rather than the ten raw **engagementStatus** values. **MAP** collapses statuses into phases. This expression goes in the Grid report's Group By field, which also accepts a Complex Expression.

```
MAP:(engagementStatus,
    'Submitted',          'Pre-Engagement',
    'Declined',           'Pre-Engagement',
    'Pending Acceptance', 'Pre-Engagement',
    'Assigned',           'Engaged',
    'Active',             'Engaged',
    'On-Hold',            'Engaged',
    'Dormant',            'At Risk',
    'Inactive',           'At Risk',
    'Abandoned',          'At Risk',
    'Completed',          'Closed',
    'Other'
)
```

Included here because Group By is the second most common Complex Expression slot operators encounter; recognizing the syntax in both Filters and Group By prevents confusion later.

7.4.5 Operator Gotchas

- **The trailing colon is mandatory. CONCAT(a, b)** will silently fail or error; it must be **CONCAT:(a, b)**.
- **Performance impact.** Complex Expressions become part of the SQL WHERE clause. Functions applied to indexed columns (e.g., **MONTH:(createdAt)**) usually prevent index use and force full table scans. For CBM data volumes this will not matter; flag it only if a report becomes slow.
- **Custom field names use the bare camelCase name.** In Complex Expressions, custom

fields do not get a **c_** prefix even though some internal metadata paths reference them with one. If the field shows as **mentoringFocusAreas** in Entity Manager, write **mentoringFocusAreas**.

- **Case sensitivity isn't intuitive.** String equality is case-insensitive by default. Use **BINARY:(...)** to force case sensitivity when it matters (rare in CBM).
- **NULL never equals anything, including NULL.** Use **IS_NULL** / **IS_NOT_NULL**, not **EQUAL:(field, NULL)**.

7.5 List-View Report Filters (Administration Feature)

The Report Filters feature exposes an existing List Report's filter logic as a primary filter on the entity's normal list view, without redefining the conditions. The list view uses its own column layout; only the filter criteria are reused.

Where it lives:

- Administration — Report Filters — Create Report Filter. Not configured inside the report itself.

How it is set up:

- Select a List Report. Its filter logic becomes the Report Filter's criteria.
- Select the entity type the filter targets — usually inferred from the report.
- Specify Teams. Members of the listed teams will see the filter as an option on the list view; everyone else will not.
- Optionally specify Roles for tighter scoping.

How it is consumed:

- As a primary filter on the entity's list view, alongside built-in primary filters such as "All" and "Assigned to me".
- As the primary filter inside a Record List dashlet.
- Inside Formula and Workflow conditions via the formula functions **record\count**, **record\findOne**, **record\findRelatedOne**, **record\findRelatedMany**, **entity\sumRelated**, and **entity\countRelated**. This is significant: a complex multi-condition filter can be defined once as a List Report and reused programmatically across the system.

Important constraint:

- The report's column layout is discarded. Only the filter conditions carry over. The list view shows the entity's normal list-view layout, not the report's columns. If the goal is a list with custom columns as well as custom filtering, run the report directly instead of using a Report Filter.

8. Front-End Configuration with clientDefs

8.1 What clientDefs Is

EspoCRM's behavior is driven entirely by JSON metadata files, grouped by **type**. Each type governs a different layer of the system. `clientDefs` is the per-entity **front-end** configuration: it controls which list and detail views render, which filters and buttons appear, which relationship panels show and how they behave, and what dynamic show/hide logic applies. It holds no business logic and no query logic — only what the browser presents.

The boundary that matters most for filtering: `clientDefs` exposes a filter in the UI, while `selectDefs` defines what that filter actually does to the query. Built-in filters such as `onlyMy` and `followed` are implemented in core, so for those you touch only `clientDefs`.

Section 7 covered filters from the Reports angle (record filters, runtime filters, and Report Filters). This section covers the metadata layer beneath the list view itself — the file that decides how the list, its filters, and its panels are presented.

The metadata types you will encounter:

Metadata type	Governs
`entityDefs`	The data model — fields, links, indexes.
`scopes`	Entity-level flags — whether it is an entity, has ACL, stream, a navigation tab.
`selectDefs`	Back-end query layer — the WHERE-clause logic for filters, default ordering, access-control filters.
`recordDefs`	Back-end service layer — save hooks, duplicate checking, link permissions.
`clientDefs`	Front-end layer — views, panels, exposed filters, buttons, menus, dynamic logic.

8.2 Where the File Lives and How Merging Works

For a customization (not a packaged extension), the file is:

```
custom/Espo/Custom/Resources/metadata/clientDefs/{EntityType}.json
```

`{EntityType}` is the internal entity name. For a custom entity built through Entity Manager, that is normally the C-prefixed name — for example a mentoring-lifecycle entity would be `CMentorship`.

Metadata is merged **recursively**: your custom file is deep-merged over the core defaults, so you declare only the keys you want to change and never restate the whole entity. A minimal file that does nothing but register the entity on the front end:

```
{
    "controller": "controllers/record"
}
```

Without `controller`, the entity type is not available in the front end at all; `controllers/record` is the built-in standard CRUD controller. After any edit, run Administration -> Clear Cache (and Rebuild if the change touches anything the back end caches), then refresh the browser.

8.3 The Parameter Landscape

The full key set is large; these are the ones reached for most often, grouped by what they govern:

Purpose	Keys

List-view behavior and filtering	`filterList`, `boolFilterList`, `defaultFilterData`, `selectDefaultFilters`, `searchPanelDisabled`, `textFilterDisabled`, `kanbanViewMode`
Views and layout	`views`, `recordViews`, `modalViews`, `additionalLayouts`, `iconClass`, `color`
Mass and row actions	`massActionList`, `massActionDefs`, `rowActionList`, `detailActionList`, `menu`, `exportDisabled`, `mergeDisabled`
Panels and relationships	`sidePanels`, `bottomPanels`, `relationshipPanels.{link}`
Dynamic behavior	`dynamicLogic`

8.4 Exposing Filters — the “Only My” Case

Any entity that has an `assignedUser` (or `assignedUsers`) field automatically surfaces the built-in **Only My** filter in the list-view filter dropdown. Selecting it shows only records assigned to the currently logged-in user, resolved dynamically per session — so one shared view works for every user, with no duplication. The selection is sticky: it persists across page refresh, logout, and re-login until the user changes it.

To declare the available bool filters explicitly (and control their order):

```
{
    "boolFilterList": ["onlyMy", "followed"]
}
```

To make a consultant **land** on their own records by default rather than relying on the sticky behavior, pre-apply the filter:

```
{
    "boolFilterList": ["onlyMy"],
    "defaultFilterData": {
        "bool": { "onlyMy": true }
    }
}
```

One behavior to know: bool filters combine with **AND**, not OR. Selecting Only My together with Followed returns only records that are both assigned to and followed by the current user.

This is distinct from Section 7’s Report Filters: those reuse a report’s logic as a primary filter, whereas `onlyMy` and `followed` are built-in toggles that need no report and no configuration beyond exposure here.

8.5 Relationship-Panel Configuration

`relationshipPanels.{link}` configures a specific related-records panel and the select dialog opened from it. To restrict the records a mentor can link from the Contacts panel on a mentoring entity to those assigned to them, and tidy the ordering:

```
{
    "relationshipPanels": {
        "contacts": {
            "selectBoolFilterList": ["onlyMy"],
```

```
            "selectPrimaryFilterName": "active",
            "orderBy": "createdAt",
            "orderDirection": "desc"
        }
    }
}
```

Note the distinction between `selectBoolFilterList` (which bool filters appear in the select modal for that panel) and `boolFilterList` (the main list view). Mixing these two up is a common error.

8.6 A Custom Filter End-to-End

For criteria beyond the built-ins — for example “assigned to me AND status = Active” — the filter spans both metadata types. The back-end `selectDefs/{Entity}.json` registers the filter and its query logic:

```
{
    "boolFilterClassNameMap": {
        "myActive": "Espo\\Custom\\Select\\CMentorship\\BoolFilters\\MyActive"
    }
}
```

That class adds a WHERE condition (assigned user = current user AND status = Active). The front-end `clientDefs/{Entity}.json` then exposes it:

```
{
    "boolFilterList": ["onlyMy", "myActive"]
}
```

Verify the PHP base-class namespace against the running EspoCRM version before writing the class; the registration pattern above is stable across versions. The takeaway: a simple “assigned to me” filter is `clientDefs` only, because it is built in; any custom criteria require `selectDefs` (the logic) plus `clientDefs` (the exposure).

No-code alternative: as covered in Section 7, the Reports feature can express many custom-criteria filters as Report Filters with current-user context and no server-side PHP. For CBM, that is usually the better path — it keeps filter logic in the administration UI rather than in deployed code.

8.7 Dynamic Logic in clientDefs

The `dynamicLogic` key declares visible / required / readOnly conditions on fields and panels — the same engine exposed in Section 5’s panel-visibility configuration and in Administration -> Dynamic Logic, expressed here in metadata:

```
{
    "dynamicLogic": {
        "panels": {
            "billing": {
                "visible": { "conditionGroup": [] }
            }
        }
    }
}
```

8.8 Mapping to CRM Builder

`clientDefs` is a pure-metadata target — no PHP — which makes it the cleanest EspoCRM layer to generate declaratively from YAML. A CRM Builder entity declaration can emit `clientDefs/{Entity}.json` from a block such as:

```
entities:
  CMentorship:
    client:
      bool_filters: [onlyMy, followed]
      default_filter:
        bool: { onlyMy: true }
      relationship_panels:
        contacts:
          select_bool_filters: [onlyMy]
          order_by: { field: createdAt, direction: desc }
```

The design point worth carrying into the generator: treat `clientDefs` and `selectDefs` as a paired unit for any custom filter, so a single declared filter emits both its query logic and its UI exposure in one operation rather than letting the two drift out of sync.

Dashboards and Dashlets

16. Dashboards and Dashlets

Every internal user in EspoCRM has a personal Dashboard at the home view, made up of dashlets arranged in a draggable grid. Dashboards are how operators see what needs their attention without navigating into individual entity list views, and how administrators distribute a standard at-a-glance layout to teams. This section covers the dashboard model, the dashlet types EspoCRM ships, the create-a-dashlet flow, the two dashlets CBM operators will reach for most often (Record List and Report), administrator-distributed Dashboard Templates, and a worked example for a per-mentor “My Active Engagements” dashlet.

16.1 Dashboards Overview

Each internal user manages their own dashboard. From the home view, a user can add new dashlets, remove existing ones, configure their options, drag the dashlet header to reorder, drag the lower-right corner to resize, and group dashlets across multiple tabs. The dashboard can be locked from the gear menu to prevent accidental edits while still allowing manual unlock.

Administrators have two routes to influence what users see by default. The first is Administration — User Interface — Dashboard Layout, which defines the system-wide default applied to every new user account. The second is Administration — Dashboard Templates, where named layouts can be assigned to individual users or to all members of a Team. Templates are starting points, not locks — a user can still rearrange their own dashboard on top of an inherited template. The administrator can reset any user’s dashboard back to the default from that user’s Preferences view, using the ‘Reset Dashboard to Default’ action.

16.2 The Dashlet Types EspoCRM Ships

EspoCRM ships a dozen built-in dashlet types out of the box, plus the Report dashlet when the Advanced Pack is installed. The full inventory:

Dashlet	Purpose	Used at CBM
Record List	Lists records of any entity type with a chosen primary filter, additional Bool filters, ordering, layout, and a max-rows cap.	Heavily. The workhorse for queue-style and personal-scope views.
Report (Advanced Pack)	Mounts a saved List or Grid report. Runtime filters surface as per-dashlet settings.	Heavily. Required for any dashlet driven by Complex Expression criteria, aggregate roll-ups, or charts.
My Activities	User's upcoming Meetings, Calls, Tasks, and event-type entities within a configurable horizon.	Useful for mentors who track scheduled Sessions and follow-ups.
Calendar	User's calendar view with selectable mode (day, week, month) and team filters.	Useful for schedule visibility.
Stream	Activity stream of records the user follows.	Useful for managers monitoring engagement progress.
My Inbox	Inbound emails for the current user.	Useful if email is configured to flow into EspoCRM.
Iframe	Embeds an external URL in a dashlet panel.	Occasional use — embedding LimeSurvey results or external monitoring pages.
Memo	Static markdown text. Useful for posting instructions to a team.	Useful for Mentor Default or Coordinator Default templates.
Sales Pipeline	Bar chart of Opportunities by stage.	Not used — CBM does not run an Opportunity entity.
Sales by Month	Bar chart of Opportunity revenue by month.	Not used.
Opportunities by Lead Source	Pie chart of Opportunities by lead source.	Not used.
Opportunities by Stage	Pie chart of Opportunities by stage.	Not used.

Custom dashlets can also be developed as JavaScript views with a small metadata JSON declaration; that is a developer-level extension and is not covered in this guide. For nearly every CBM dashboard use case, Record List and Report cover the ground.

16.3 Creating a Dashlet on Your Personal Dashboard

From the home view, the dashboard editing controls are reached through the gear icon at the top right of the dashboard area, or through the '+' button on a tab strip. The full add-dashlet flow:

- **Step 1 —** Open the home view (the icon at the top left of the EspoCRM navigation bar).
- **Step 2 —** Open the dashboard gear menu and choose 'Add Dashlet' (or click the '+' on a tab to add a dashlet to that tab). The dashlet type picker appears.
- **Step 3 —** Pick a dashlet type. The dashlet is created with default options and appears immediately on the dashboard.

- **Step 4 —** Click the new dashlet's gear icon and choose Edit. The options form opens.
- **Step 5 —** Fill in the options. Configuration fields are specific to the dashlet type — 16.4 covers Record List, 16.5 covers Report. Save.
- **Step 6 —** Drag the dashlet header to reorder it on the grid. Drag the lower-right corner to resize. Use the dashlet gear menu for Edit, Remove, and Refresh actions.

To prevent accidental edits, the dashboard gear menu has a 'Lock Dashboard' option. Locked dashboards still allow manual unlock from the same menu.

16.4 The Record List Dashlet

Record List is the most common dashlet type at CBM. It lists records of a chosen entity type with the entity's normal list-view layout, scoped by filters, ordered as specified, capped at a chosen number of rows.

Configuration options:

Option	Purpose
Title	Display label shown on the dashlet header. Usually phrased as the question the dashlet answers, e.g., "My Active Engagements".
Entity Type	Which entity type the dashlet lists. Any entity the user has Read access to is eligible.
Primary Filter	A built-in primary filter (such as Open, All) or a custom Report Filter from Administration — Report Filters (see 7.5). Determines the principal scope of the result set.
Additional Filters	The Bool filters available for this entity — for example, Only My (records assigned to the current user) and Followed (records the user is following). Each Bool filter is a checkbox; multiple checked filters combine with AND. Behaviour and the clientDefs configuration that exposes these filters on custom entities are covered in 8.4.
Order By and Order	The field to sort by and the direction. Defaults to the entity's normal list-view sort.
Layout	Which list layout the dashlet uses, from the layouts defined in Administration — Entity Manager — [Entity] — Layouts — List. If only one list layout is defined, this option is fixed.
Max Number of Records	The row cap. Common values: 10, 20, 50. Larger values increase render time.
Auto-Refresh Interval	How often the dashlet re-fetches its data, in minutes. Options include 0 (manual refresh only), 0.5, 1, 2, 5, and 10. Stream-sensitive views benefit from 1 or 2; static reference lists can stay at 0.

Cross-reference for Only My on custom entities. The Only My Bool filter is available on built-in CRM entities by default. For custom entities such as Engagement, Only My must be exposed by adding the entity's clientDefs metadata file (see 8.4 for the one-line edit and a cache-clear step). Once exposed, Only My appears as an Additional Filter on the Record List dashlet, on the entity's standard list view, and on any future dashlet on that entity — not just this one.

16.5 The Report Dashlet

The Report dashlet is available when the Advanced Pack is installed. It mounts a saved Report

(List or Grid) on the dashboard, with any of the report's runtime filters appearing as per-dashlet options. This is the path for dashlet-mounted aggregates, charts, and any filter logic that would exceed what the Record List dashlet's built-in primary and Bool filters can express.

Configuration options:

Option	Purpose
Title	Display label on the dashlet header. Often inherits from the report name.
Report	A picker showing every List Report and Grid Report the user has Read access to. The chosen report defines the columns, the underlying record filter, the chart type (for Grid reports), and any runtime filters.
Runtime Filter values	Each runtime filter the report defines surfaces here as an editable option. Different users mounting the same report can supply different runtime values without modifying the report itself (see 7.3 for the runtime-filter usage modes).

List vs Grid rendering. A List Report renders as a tabular list of records, similar in shape to a Record List dashlet but with the report's chosen columns and ordering rather than the entity's list layout. A Grid Report renders as a chart when grouped on a single column (bar, pie, line, or whatever the report specifies) and as a table when grouped on two columns.

When to pick Report over Record List:

- The criteria require a Complex Expression filter (see 7.4) that a Record List dashlet's built-in filter combinations cannot express.
- The output is a chart or aggregate roll-up rather than a flat record list.
- Different users or different dashboards need different runtime values for the same underlying filter, supplied per dashlet at mount time.
- Custom column selection or column ordering distinct from the entity's list-view layout is needed.

16.6 Dashboard Templates and Distribution

Administrators distribute dashboard layouts through two mechanisms:

- **System default.** Administration — User Interface — Dashboard Layout defines the layout new user accounts inherit at creation. Existing users are not retroactively reset unless explicitly reset from their Preferences.
- **Dashboard Templates.** Administration — Dashboard Templates creates named, reusable layouts. Each template can be assigned to a specific user (overrides the system default for that user) or to all members of a Team (overrides the system default for every team member). A template is a starting point: once applied, the user can still modify their dashboard on top of it. The administrator can reset any user back to the template (or the system default) from that user's Preferences view.

CBM application. Distinct templates per role are the natural pattern: a Mentor Default template carrying the My Active Engagements dashlet (built in 16.7), the My Activities dashlet for upcoming Sessions, and a Stream dashlet on records the mentor follows; a Coordinator Default template carrying the Coordinator Stale-Access Dashlets referenced in 13.10 alongside an All-Active-Engagements Record List for at-a-glance pipeline visibility. Both templates are assigned to the corresponding Team rather than to individual users, so new team members inherit the right layout automatically.

16.7 Worked Example: My Active Engagements

Goal: a Record List dashlet that shows, on each mentor's home page, only the engagements where they are the assigned mentor and the engagement is currently in Active status. The same dashlet definition serves every mentor — the per-user scoping resolves dynamically at view time.

Prerequisites:

- Advanced Pack installed (required for Report Filters — see 7.5).
- Engagement's assignedUser populated when the mentor is matched to the engagement. The CBM design (see 14 and the Mentoring domain PRDs) copies the assigned mentor's linked User into the Engagement's assignedUser at the Assigned-status transition.

Path B — Recommended — UI only, no server access required.

Builds a List Report with a Complex Expression filter that scopes to the current user dynamically, saves it as a Report Filter (see 7.5), and mounts that Report Filter as the Primary Filter of a Record List dashlet.

Step 1 — Create the underlying List Report.

- Administration — Reports — Create Report.
- Type: List.
- Target Entity: Engagement.
- Name: "Active Engagements (Assigned to Me)".
- On the Filters tab, add two filters:
- **Field filter:** Engagement Status Equals Active.
- **Complex Expression filter:** see the values below.

```
Complex Expression:  assignedUserId
Operator:            Equals
Formula:             env\userAttribute('id')
```

- On the Columns tab, pick the columns shown in the dashlet (Name, Account, Engagement Status, Last Session Date, Meeting Cadence are the usual set).
- Save the report.

Step 2 — Create the Report Filter.

- Administration — Report Filters — Create.
- Report: select "Active Engagements (Assigned to Me)".
- Teams: Mentors (or every team whose members should see this filter).
- Save.

Step 3 — Mount as a Record List dashlet on the Mentor Default dashboard template.

- Administration — Dashboard Templates — Mentor Default (or create the template per 16.6 if it does not exist yet).
- Add a dashlet (16.3 covers the flow).
- Type: Record List.
- Title: "My Active Engagements".

- Entity Type: Engagement.
- Primary Filter: select the new “Active Engagements (Assigned to Me)” Report Filter.
- Order By: Last Session Date, descending (so the most-recently-active engagements appear first).
- Max Number of Records: 20.
- Save the dashlet, then save the template.

Result: every mentor whose team is assigned the Mentor Default template sees the dashlet on their home page, populated with only their own engagements that are currently in Active status. The env\userAttribute('id') call resolves to the viewing user’s ID at query time, so a single Report Filter definition correctly scopes per user with no duplication and no maintenance.

Path A — Alternative when “Only My” should be available everywhere on Engagement, not just in this dashlet.

If the Only My Bool filter is enabled on Engagement (a one-time clientDefs edit and Cache Clear documented in 8.4), the worked example simplifies. The underlying Report Filter only needs the status condition — the per-user scoping is provided by the dashlet’s Additional Filters rather than by a Complex Expression in the Report. Build the Report Filter with a single field filter (Engagement Status Equals Active), mount a Record List dashlet on Engagement with that Report Filter as the Primary Filter, and check Only My in the Additional Filters. The result is identical to Path B. The advantage is that Only My then also appears on the regular Engagement list view, on every related panel, and on any future Engagement dashlet — not just this one.

When to pick which. Path B requires only administration-UI access and no server file-system access; it is the default recommendation for sites where the clientDefs edit is not available. Path A is preferred where server access is available and the operator wants the Only My pattern to apply across every Engagement view, not only this dashlet. The two paths are not mutually exclusive: Path A unlocks Only My everywhere, and a Report Filter built with the Complex Expression of Path B remains available as a primary filter for any caller that wants the combined criteria in a single, dropdown-selectable filter.

Security Configuration

12. EspoCRM Security Concepts

This section explains how EspoCRM’s access control system works in general terms, independent of any particular implementation. The concepts here apply to every EspoCRM deployment and are prerequisites for understanding the CBM-specific configuration in Section 13.

12.1 Overview

EspoCRM security answers one question, repeatedly: can this user perform this action on this record? Four inputs determine the answer — who the user is, what teams they belong to, what roles they have, and what data is on the record itself (assignedUser, teams, and field values).

One escape hatch is worth noting up front: System Administrators bypass all access control. When testing permissions, never test as admin — you will see everything regardless of how the

role configuration is set. Always test with a regular user account, ideally in an incognito browser window.

12.2 The Three Building Blocks: Users, Teams, Roles

EspoCRM access control is built from three primitives that interact:

Users are individual accounts. Each user has a primary team, optional additional teams, a default assigned user (usually themselves), and one or more roles. Users can also have a default team set — any record they create is automatically stamped with those teams.

Teams are groupings of users. They serve two purposes: record-level access keys off of team membership, and roles can be attached to teams so every member inherits those roles. A user can belong to many teams, and a record can be assigned to many teams.

Roles are the permission definitions. A role is essentially a matrix: for each entity (Contact, Account, Meeting, your custom entities), what actions are allowed and at what level.

12.3 The Action × Level Matrix

For every entity, a role defines five actions: Create, Read, Edit, Delete, and Stream. Create is binary (yes/no). The other four each take one of four levels:

Level	What It Means
All	Any record of this entity, regardless of team or ownership.
Team	Only records where the user shares at least one team with the record.
Own	Only records where the user is the assignedUser (or in assignedUsers if Multiple Assigned Users is enabled).
No	Never.

Example: Read=Team, Edit=Own on the Contact entity means “I can see every Contact in my team’s pipeline, but I can only edit Contacts assigned to me.”

12.4 Field Level Security

Beneath the entity-level matrix, you can lock down individual fields within an entity, independently for read and edit. Everyone might be able to read a Contact, but only managers see the salaryExpectation field. Field-level rules can only further restrict what the entity-level matrix already permits — they can never grant access that the entity-level denies.

Field-level security applies to link fields too — the field that represents a relationship between two entities. Hiding a link field for a role makes the corresponding related-record panel invisible on the detail view. This is the mechanism used to hide sensitive relationship panels, described in Section 12.9.

Important limitation: field-level security is per-role, not per-record. You cannot configure “User X sees the bankBalance field on Acme but not on Beta.” If per-record field control is needed, the workaround is to split the sensitive fields into a separate entity — see Section 12.10.

12.5 Assignment Permission

Assignment Permission is a system-level permission in the Permissions section of a Role (separate from the entity scope matrix and field-level rules). It controls who the user can set as assignedUser, assignedUsers, teams, or collaborators on any record. It has three levels:

Level	Effect

No	The user cannot modify any field that controls record ownership or access. They can edit the data fields of a record but the assignedUser/assignedUsers/teams/collaborators fields appear read-only.
Team	The user can only assign to other users who share at least one team with them.
All	The user can freely modify ownership/access fields, assigning to any user.

Assignment Permission is enforced separately from regular Edit access. A user can have Edit=Own on a record — meaning they can change the data fields — without being able to touch the assignedUsers or teams fields. This is the safeguard that prevents an ordinary user from quietly reassigning ownership or granting access to others.

12.6 Multiple Assigned Users vs Collaborators

By default an EspoCRM record has a single assignedUser. Two optional features change this:

Multiple Assigned Users — enabled per-entity in Entity Manager. Replaces the single assignedUser field with an assignedUsers link-multiple field. Every user in the list has Own-level access (both Read and Edit, governed by the role's Own settings). Use this when several users need full edit access to the same record.

Collaborators — also enabled per-entity in Entity Manager. Creates a link-multiple field called Collaborators. Users added as collaborators get read and stream access (not edit) to the record, provided their role permits anything other than No. The creator of a record and the current assigned user are auto-added as collaborators. Adding a user as a collaborator requires Assignment Permission and Edit access to the record. Portal users cannot be added as collaborators.

When the authorized users need to edit the record, choose Multiple Assigned Users. When they only need to view it, Collaborators is lighter. After enabling either feature, the field must be added to the entity's side panel layout (Administration > Entity Manager > {Entity} > Layouts > Side Panel Fields) before it becomes visible.

12.7 Role Merging

A user can have multiple roles, assigned directly through their User record or inherited through Team membership. When a user has multiple roles, EspoCRM merges them by taking the most permissive level for each setting. If one role says Read=Own and another says Read=All, the user ends up with Read=All. Roles are additive, not restrictive.

This rule matters: you cannot use a second role to subtract permissions that a first role granted. To withhold access from a subset of users, configure their primary role to withhold it and selectively layer a second role on those who should have it.

To check what permissions a specific user has, open their User record and click the Access button — EspoCRM shows the merged effective permissions.

12.8 Roles via Team Attachment vs Direct Assignment

Roles can be assigned to users in two ways:

Direct: Administration > Users > select user > Roles field > add the role. The user gets the role regardless of team membership.

Via team attachment: Administration > Teams > select team > Roles field > add the role. Every user in that team automatically inherits the role. Add or remove a user from the team and their role membership follows.

Team-attached roles are usually cleaner for groups of users who share a function (“Mentors,” “Data Integrity Team,” “Sponsor Coordinators”). Membership management lives in one place: add a person to the team and they get the role; remove them and they lose it. Direct role assignment is appropriate for unique grants that don’t correspond to a group identity (e.g., one specific user who is a Coordinator).

12.9 Hiding Relationship Panels Using Field-Level Security

When two entities are linked through a relationship, EspoCRM creates link fields on both sides. These link fields appear as related-record panels on the detail view. Because link fields are subject to field-level security like any other field, setting field-level read access to No on a link field hides the panel entirely for that role.

When the link field is hidden for a role, the related-records panel does not render on the detail view, the field is omitted from API responses, and it does not appear in search or filter pickers. The user sees no trace that a related record could exist. This is the central trick for hiding sensitive related-record panels from users who should not even know the related data exists.

To configure: open the role, scroll to Field Level Permissions, click the plus icon next to the entity that owns the link field, select the link field from the dropdown, and set Read=No and Edit=No.

12.10 The Sensitive Sub-Entity Pattern

This is a reusable design pattern for a specific challenge: a small subset of records contain sensitive data that should be visible only to a specific list of users, where the authorized list varies per record. EspoCRM’s field-level security cannot do per-record restrictions, so the pattern works around the limitation by splitting the sensitive data into a separate entity:

- Step 1: Create a separate entity (e.g., ClientProfile) to hold the sensitive data, linked one-to-one to the parent entity.
- Step 2: Enable Multiple Assigned Users (or Collaborators) on the sensitive entity, so the per-record authorized list can be expressed.
- Step 3: Create a baseline “Standard User” role that has the sensitive entity disabled in the scope matrix, and field-level read=No on the link field from the parent entity to the sensitive entity. This hides both the entity and the related-records panel for users without sensitive-data access.
- Step 4: Create an “Authorized” role that adds scope Read=Own and Edit=Own on the sensitive entity, and unhides the link field at field level. Layer this role on top of Standard User for users who can be granted access to one or more sensitive records.
- Step 5: Optionally create a “Coordinator” role with scope Read=All and Edit=All on the sensitive entity, plus Assignment Permission=All. Coordinators administer the sensitive records and decide who else gets access by editing the assignedUsers field.

The pattern is reusable for any case requiring per-record dynamic access lists. CBM uses it three times in parallel for its three sensitive sub-entities; see Section 13.

13. CBM Security Implementation

This section documents the specific security configuration CBM uses, built on the concepts in Section 12. It is the consolidated specification that future administrators should consult when adding users, defining new sensitive entities, or troubleshooting access issues.

13.1 Architecture Summary

One Company entity is universally visible to all users. Three sensitive sub-entities (ClientProfile, SponsorProfile, PartnerProfile) hold restricted data, each linked one-to-one to Company. Each sensitive sub-entity is hidden by default through field-level security on its corresponding link field on Company, and visible only to users on the corresponding access team.

Per-team users get automatic ownership-derived access to sensitive records for Companies they own — a workflow rule synchronizes the Company.assignedUser into the sensitive record's assignedUsers field. They can also transfer access among their peers via team-scoped Assignment Permission. Per-type Coordinators administer their respective sensitive entities. A Data Integrity team handles Company-level data quality. The Admin role retains universal access.

13.2 Entity Setup

The Company entity (extension of the native Account entity) is the parent. Three sensitive sub-entities are linked to it:

Sensitive Entity	Link Field on Company	Multiple Assigned Users
ClientProfile	clientProfile	Enabled
SponsorProfile	sponsorProfile	Enabled
PartnerProfile	partnerProfile	Enabled

Each relationship is configured as One-to-One Right with Company as the parent side. Multiple Assigned Users is enabled on all three sensitive entities so that the per-record authorized list can hold multiple users — all of whom get Own-level access.

13.3 Teams

Four teams structure the access pools. Each access team has a role attached so that membership automatically grants the corresponding role:

Team	Role Attached to Team	Purpose
Mentors	Client Sensitive Authorized	Users who can be granted access to ClientProfile records (typically as mentors of Client Companies).
Sponsor Coordinators	Sponsor Sensitive Authorized	Users who manage sponsor relationships and need access to SponsorProfile records.
Partner Managers	Partner Sensitive Authorized	Users who manage partner relationships and need access to PartnerProfile records.
Data Integrity Team	Data Integrity	Users who can edit any Company record for data quality purposes.

13.4 Standard User Role

Every user is assigned this role. It provides baseline access to Company records and completely hides all three sensitive entities.

Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
Company	enabled	yes	All	Own	No	All
ClientProfile	disabled	—	—	—	—	—
SponsorProfile	disabled	—	—	—	—	—
PartnerProfile	disabled	—	—	—	—	—

Field Level Permissions on Company:

Field	Read	Edit
clientProfile	No	No
sponsorProfile	No	No
partnerProfile	No	No

Permissions: Assignment Permission = No.

13.5 Sensitive Authorized Roles

Three parallel roles, one per sensitive entity. Each is attached to the corresponding access team so that adding a user to the team grants the role automatically. Each role grants Read=Own and Edit=Own on its specific sensitive entity, and unhides the corresponding link field on Company.

The three roles are structurally identical, differing only in which sensitive entity they target. The Client Sensitive Authorized role configuration is shown below; Sponsor Sensitive Authorized and Partner Sensitive Authorized are configured the same way, substituting SponsorProfile / sponsorProfile and PartnerProfile / partnerProfile respectively.

Client Sensitive Authorized — Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
ClientProfile	enabled	no	Own	Own	no	Own

Field Level Permissions on Company:

Field	Read	Edit
clientProfile	Yes	—

Permissions: Assignment Permission = Team. This allows team members to add or remove fellow team members from the assignedUsers field of a ClientProfile, enabling peer-to-peer handoff without Coordinator involvement.

13.6 Sensitive Coordinator Roles

Three parallel coordinator roles, one per sensitive entity. Assigned directly to users via their User record (not through team attachment, because the coordinators are a small group with elevated privileges and team semantics don’t apply). The Master Coordinator function — access to all sensitive entities — is served by the Admin role, not a separate Master Coordinator role.

Client Sensitive Coordinator configuration is shown below; Sponsor and Partner Coordinator roles follow the same pattern for their respective entities.

Client Sensitive Coordinator — Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
ClientProfile	enabled	yes	All	All	All	All

Field Level Permissions on Company:

Field	Read	Edit
clientProfile	Yes	Yes

Permissions: Assignment Permission = All. Required so the Coordinator can modify the assignedUsers field on any ClientProfile to grant or revoke access for specific users.

13.7 Data Integrity Role

Attached to the Data Integrity Team. Provides Edit=All on Company so that members can correct errors anywhere in the Company directory. The role does not grant access to any sensitive entity; team members who also need sensitive access must additionally hold the relevant Sensitive Authorized or Coordinator roles.

Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
Company	enabled	not-set	not-set	All	not-set	not-set

Permissions: Assignment Permission = No. Data Integrity Team members can edit any Company's data fields but cannot reassign Company ownership. Ownership changes require Coordinator or Admin involvement.

13.8 User Assignment Patterns

Roles and teams compose additively. The table below summarizes how to configure each type of user; combinations are valid (e.g., a user can be both a Mentor and a Partner Manager by being in both teams).

User Type	Direct Role Assignments	Team Memberships
Standard staff	Standard User	(none)
Mentor	Standard User	Mentors
Sponsor Coordinator	Standard User	Sponsor Coordinators
Partner Manager	Standard User	Partner Managers
Data Integrity volunteer	Standard User	Data Integrity Team
Per-type Coordinator	Standard User + the relevant Coordinator role	(optional access team)
Multi-role user	Standard User	All relevant access teams
Administrator	Admin	(bypasses ACL entirely)

13.9 Workflow Rules

Three independent workflow rules synchronize Company ownership into the corresponding sensitive entity's assignedUsers field. Each rule is additive only — it adds the current Company.assignedUser to the related sensitive record's assignedUsers when Company ownership changes, but never removes anyone. Stale access is detected through the

Coordinator Stale-Access Dashlets described in Section 13.10.

Workflow Rule	Trigger	Action
Sync Mentor to ClientProfile	Company.assignedUser change (or ClientProfile created)	Add the current Company.assignedUser to ClientProfile.assignedUsers.
Sync Sponsor Coordinator to SponsorProfile	Same on the corresponding Company/SponsorProfile pair	Same, targeting SponsorProfile.assignedUsers.
Sync Partner Manager to PartnerProfile	Same on the corresponding Company/PartnerProfile pair	Same, targeting PartnerProfile.assignedUsers.

13.10 Coordinator Stale-Access Dashlets

Each Coordinator role has a corresponding dashlet on their home dashboard, showing sensitive entity records where the assignedUsers list contains a user who is not the current Company.assignedUser. These records have stale access — typically the result of Company reassignment without follow-up cleanup. The Coordinator reviews the list on a regular cadence and removes outdated entries from assignedUsers as appropriate.

This design choice keeps revocation explicit and auditable: a person decides to remove access, rather than the workflow silently revoking it on reassignment. The trade-off is operational discipline — the Coordinator must do the cleanup.

13.11 Test Plan

After any change to the security configuration, the following test users should be verified by logging in as each one (in an incognito window — never test as Admin) and confirming the access matches expectations:

test_standard — Standard User only. Should see all Companies but no ClientProfile, SponsorProfile, or PartnerProfile panels anywhere.

test_mentor — Standard User + Mentors team. Set as Company.assignedUser on at least one Company with a ClientProfile, and not on a different Company that also has a ClientProfile. Should see the ClientProfile panel populated on the owned Company and empty on the non-owned Company.

test_dual_mentor_partner — Standard User + Mentors team + Partner Managers team. Should see ClientProfile panels and PartnerProfile panels independently, but no SponsorProfile panels.

test_client_coord — Standard User + Client Sensitive Coordinator role. Should see all ClientProfile panels and records, regardless of Company ownership.

test_data_integrity — Standard User + Data Integrity Team. Should be able to edit any Company’s data fields, but should not see any sensitive entity panels.

This section will document how to configure user roles and permissions in EspoCRM to control what actions users can perform on records and relationship panels.

Topics planned for this section — pending research:

- 7. Restricting the Remove function in relationship panels — preventing users from permanently deleting linked records via the panel
- 8. Restricting the Unlink function in relationship panels — controlling whether users can remove associations between records

Note: See the Questions for Research section for the open questions that need to be resolved before these topics can be fully documented.

14. User Auto-Assignment

The assignedUser field is the foundation of Own-level access in EspoCRM. For the Sensitive Sub-Entity Pattern in Section 12.10, and for any role using Edit=Own or Read=Own, this field must reliably be populated when records are created. This section describes the three layers of behavior that govern automatic assignment and how to ensure it happens consistently across UI, API, import, and workflow.

14.1 Default Behavior

By default, when a user opens the create form for any entity that has the assignedUser field, EspoCRM pre-fills that field with the current user. This is built-in behavior and requires no configuration. The user can override the value before saving if they have Assignment Permission, but the pre-fill always happens automatically on the create form.

14.2 The User Preference That Can Disable It

Each individual user has a preference called “Do not pre-fill Assigned User on record creation” in their User Profile under Preferences. When this checkbox is ticked for a user, the auto-pre-fill is suppressed for that user only — they must manually select an assignedUser before saving (or leave it blank, if the field is not required).

Symptom: a user reports that records they create do not have themselves as assigned user. Diagnosis: this preference is likely checked on their account. Fix: open their User record, go to Preferences, and uncheck “Do not pre-fill Assigned User on record creation.”

14.3 Bulletproofing with a Before Save Formula

Both the default behavior and the user preference operate at the UI layer. They affect what is pre-filled on the create form, but they do not enforce anything at the database layer. Records created through the API, through import, or through a workflow rule do not go through the UI pre-fill logic at all, and the assignedUser field may end up empty.

For guaranteed enforcement at the database layer, configure the entity’s Before Save formula in Entity Manager. Navigate to Administration > Entity Manager > {Entity} > Formula and add:

```
if (entity\isNew() && assignedUserId == null) {
    assignedUserId = env\userAttribute('id');
    assignedUserName = env\userAttribute('name');
}
```

This formula fires on every record create regardless of the source (UI, API, import, workflow). The entity\isNew() check restricts the action to record creation. The assignedUserId == null check ensures that if an explicit assignment was already made (for example by a Coordinator deliberately assigning to someone else, or by a workflow that already set the value), the formula leaves that choice in place.

14.4 Multiple Assigned Users

Entities that have the Multiple Assigned Users feature enabled (Section 12.6) use the plural field assignedUsersIds, a list of user IDs. The plural field does not always auto-populate the creator the same way the singular assignedUser field does. If creator inclusion in the access list is

important, the safe approach is a Before Save formula that explicitly adds the current user:

```
if (entity\isNew() && !array\includes(assignedUsersIds, env\
userAttribute('id'))) {
    assignedUsersIds = array\push(assignedUsersIds, env\
userAttribute('id'));
}
```

The array\includes check prevents duplicate entries if the user is already in the list from another mechanism (a workflow, a default value, an explicit form entry). The array\push adds the user without disturbing other entries the workflow or form already populated.

14.5 Default Team Set

EspoCRM offers a related mechanism for teams. Each user can have a Default Team Set configured on their User record. When the user creates any record, the teams listed in their Default Team Set are automatically added to that record's teams field. This auto-population is the team equivalent of assignedUser pre-fill: a convenience that ensures team-based access (Read=Team, Edit=Team) works without manual configuration on every record.

Configure under Administration > Users > {user} > Default Teams (the exact label varies slightly by EspoCRM version). The teams listed here will be applied to every record this user creates, unless explicitly overridden on the create form.

14.6 Application to CBM

For CBM's security model the recommended configuration is as follows.

Company entity (single assignedUser). The default pre-fill behavior in Section 14.1 is sufficient for the typical case. All users have "Do not pre-fill Assigned User" unticked. Optionally add the Section 14.3 Before Save formula to bulletproof against records created via API or import paths.

ClientProfile, SponsorProfile, PartnerProfile (Multiple Assigned Users). The workflow rules described in Section 13.9 synchronize the Company.assignedUser into the sensitive entity's assignedUsersIds when Company ownership is set or changed. The creating Coordinator does not need to be added to assignedUsersIds automatically, because their Coordinator role grants Read=All and Edit=All on the sensitive entity regardless of the assignedUsers list. If belt-and-suspenders enforcement is desired, add a Section 14.4 formula on each sensitive entity to ensure the current Company.assignedUser is captured at creation time even if the workflow has not yet fired.

Default Team Set. If CBM adopts the team-per-pairing model discussed in earlier security design sessions, the Default Team Set on each Mentor user can pre-populate the relevant pairing team on records they create. This is a convenience optimization, not a requirement.

Integration Configuration

9. Integrating Google Meet with EspoCRM Meetings

EspoCRM does not create Google Meet links directly. Instead, it syncs meetings to Google Calendar, and Google automatically generates a Meet link as part of that calendar event — provided the user's Google Calendar is configured to do so. No manual action is needed in EspoCRM for the Meet link itself.

9.1 How It Works

When a meeting is created in EspoCRM and synced to a user's Google Calendar, Google Calendar automatically attaches a Meet link to the event if the user's calendar has Meet link generation enabled. This is standard behavior for Google Workspace accounts.

The flow is:

24. A meeting is created in EspoCRM and assigned to a user.
25. EspoCRM syncs the meeting to that user's Google Calendar via the Google Calendar sync module.
26. Google Calendar automatically generates and attaches a Meet link to the event.
27. The Meet link is visible in the Google Calendar event, and attendees receive it in their calendar invitation.

Note: EspoCRM does not currently display the Google Meet link back inside the EspoCRM meeting record. Users will need to open the event in Google Calendar to access the Meet link, or find it in their calendar invitation email.

9.2 Prerequisites

Before this integration will work, the following must be in place:

- Google Calendar sync must be configured in EspoCRM. Navigate to: Admin → Extensions → Google Calendar Sync to confirm it is set up.
- Each user must have their personal Google account connected to EspoCRM via the Google Integration (Admin → Integrations → Google).
- The user's Google Calendar must be configured to automatically add Meet links to events. This is a Google Calendar setting, not an EspoCRM setting — see Section 9.3 below.

9.3 Enabling Automatic Meet Links in Google Calendar

This setting is configured in Google Calendar, not in EspoCRM. Each user should confirm this is enabled in their own Google Calendar account:

28. Open Google Calendar in a browser and click the Settings gear icon.
29. Go to Settings → General → Event Settings.
30. Ensure the option to automatically add video conferencing to events is enabled.
31. Save settings.

For Google Workspace organizations (such as CBM using Google Workspace for Nonprofits), this setting may also be controlled at the admin level via the Google Workspace Admin Console

under Calendar settings.

9.4 Creating a Meeting in EspoCRM

Once the prerequisites are in place, creating a meeting that generates a Meet link requires no special steps:

32. Create a new Meeting record in EspoCRM (via the Meetings entity or from within a related record such as a Contact or Engagement).
33. Assign the meeting to the appropriate user and set the date and time.
34. Save the meeting. EspoCRM will sync it to Google Calendar automatically (sync may take a few minutes depending on sync frequency settings).
35. Open the event in Google Calendar to find the Meet link. Attendees listed on the EspoCRM meeting will receive a calendar invitation with the Meet link included.

Note: Sync frequency is controlled by the Google Calendar sync configuration. If the Meet link does not appear immediately, wait a few minutes and refresh the Google Calendar event.

10. Integrating EspoCRM Contacts with Google Contacts

EspoCRM can push Contact and Lead records from EspoCRM into Google Contacts, keeping your CRM data accessible from any Google service or device that syncs with Google Contacts.

Note: Before setting up Google Contacts integration for users, the administrator must first configure the Google Integration at the system level. Confirm this has been completed before proceeding with the user-level setup below.

10.1 What Gets Pushed

The following fields are pushed from EspoCRM to Google Contacts when a sync is performed:

- Name
- Description
- Email Addresses
- Phone Numbers
- Account Name
- Account Title
- Address

10.2 User Setup

Each user must connect their Google account to EspoCRM and enable the Google Contacts scope before they can push records. To do this:

36. Go to your user detail view by clicking your name in the top-right menu.
37. Click the External Account button.
38. Click Google in the left panel.
39. Check the Enabled checkbox, then click the Connect button.
40. A popup will appear asking for your consent to grant EspoCRM access to your Google account. Approve it.
41. A green Connected label should appear confirming the connection was successful.
42. Check the Google Contacts checkbox to enable contact syncing.
43. Optionally, select one or more Google Contacts Groups to specify which group(s) EspoCRM records will be pushed into.
44. Click Save.

Note: If the Google Contacts checkbox does not appear after connecting successfully, the administrator has not granted access to the Google Contacts scope. Contact your administrator to enable it.

10.3 Pushing Contacts to Google

Once the user setup is complete, contacts and leads can be pushed to Google Contacts manually:

45. Go to the Contacts or Leads list view.
46. Select the records you want to push using the checkboxes.
47. Click the Actions dropdown and select Push to Google.

A single push action can process up to 10 records immediately. If more than 10 records are selected, the remainder will be pushed in the background by the system's scheduled task (cron). After the push completes, the contacts will be available in Google Contacts, organized into the groups specified during setup if applicable.

11. SMS Sending via Twilio

EspoCRM includes built-in functionality for sending SMS messages but does not ship with any SMS provider out of the box. SMS sending requires the official SMS Providers extension, which adds support for multiple providers including Twilio.

11.1 Supported SMS Providers

The SMS Providers extension supports the following providers:

- Twilio
- Spryng
- seven
- smstools
- SerwerSms
- Verimor
- GatewayAPI

> **Note:** Out of the box, SMS can be sent from a Formula script or PHP code. The VoIP Integration additionally provides the ability to send SMS through the EspoCRM UI, but this is available only for the Twilio provider.

11.2 Step 1 — Install the SMS Providers Extension

Download the extension package from the EspoCRM SMS Providers GitHub repository (look under the Assets section of the latest release). Then install it in EspoCRM:

48. Go to: Admin → Extensions.
49. Click Choose File and select the downloaded extension package.
50. Click Upload, then follow the prompts to install.

11.3 Step 2 — Configure the SMS Provider

After the extension is installed, configure the SMS provider settings:

51. Go to: Admin → SMS.
52. Select your SMS provider from the Provider dropdown (e.g., Twilio).
53. In the SMS From Number field, enter the sending phone number including the country code (e.g., +11111111111).
54. Click Save.

55. Go to: Admin → Integrations and select your provider (e.g., Twilio).
56. Enter the required credentials and parameters for that provider (for Twilio: Account SID and Auth Token from your Twilio console).
57. Click Save.

11.4 Sending SMS via Formula Script

Once the extension is installed and configured, SMS messages can be sent programmatically

using the ext\sms\send formula function. This is useful for automated notifications triggered by workflows or other system events.

11.5 Sending SMS via Workflow (Requires Advanced Pack)

With the Workflow tool and the Advanced Pack extension, EspoCRM can be configured to send SMS notifications automatically when specific conditions are met. To set this up:

58. Create a new Workflow rule with the desired trigger type and conditions (e.g., when an Engagement status changes to Active).
59. Add an Execute Formula Script action to the workflow.
60. Paste a formula script that creates an SMS record and sends it. A basic example:

```
$body = 'Hi! This is an SMS notification from EspoCRM.';

// Phone number is obtained from the entity.
$phoneNumber = phoneNumber;

$smsId = record\create(
    'Sms',
    'to', $phoneNumber,
    'body', $body
);

ext\sms\send($smsId);
```

Note: The Advanced Pack extension is required to use Workflows in EspoCRM. Confirm this extension is installed before attempting to configure workflow-based SMS sending.

Automation Configuration

15. Formulas and Calculated Fields

EspoCRM provides several mechanisms for automatically computing field values: Before Save formulas attached to an entity, calculated values produced by Workflow actions, and the formula functions that operate inside both. This section explains how each mechanism works, when to use which, and how they combine to keep fields such as record counts and rolled-up totals accurate over time.

15.1 Overview

A formula in EspoCRM is a short script written in EspoCRM's built-in formula language. Formulas can read and write fields on the current record, traverse relationships, count or sum related records, and call a library of built-in functions. Formulas appear in two operational contexts:

- Before Save Formula on an entity — configured at Administration → Entity Manager → [Entity] → Formula. Runs on every save of a record of that entity, regardless of whether the save came from the UI, the API, an import, or a workflow. This is where formulas that

compute fields on the host record live.

- Workflow action formulas — configured inside Workflow rules (Advanced Pack), most commonly in Update Target Record and Update Related Record actions. These formulas calculate values to write onto either the workflow's own target record or a related record reached through a link.

A calculated field is any field whose value is set by a formula rather than typed in by a user. The field itself is a normal field (Integer, Float, Varchar, Date, and so on) defined in Entity Manager. What makes it calculated is that one of the mechanisms above writes to it. There is no separate "Calculated" field type. Marking the field as Read-only in Entity Manager is the conventional way to prevent users from overwriting the formula's output.

15.2 When to Use a Formula vs a Workflow vs Both

The choice depends on what triggers the recalculation. A Before Save formula recomputes whenever the host record is saved — but if the value depends on related records that change independently (a child being added, a sibling being unlinked), the host record may not be saved when those changes happen, and the field goes stale. A Workflow on the related (child) entity can detect those child-side changes in real time and push the recomputed value back to the parent.

The decision framework that follows covers the three common shapes of calculated-field problem in CBM:

Calculation depends on...	Use	Why
Only fields on this record	Before Save formula on this entity	The save itself is the trigger — every save recomputes the value, every field is current.
Related records that change with the host	Before Save formula on this entity	Related-record changes typically pass through a save of the host, so the formula sees the latest state.
Related records that change independently of the host	Workflow on the related entity, plus Before Save formula on this entity	The workflow keeps the value fresh as children change. The Before Save formula handles backfill of existing records and protects against missed workflow runs.

15.3 Before Save Formulas on an Entity

A Before Save formula is the simplest place to compute a field value. It runs on every save of the host record — create or update, from any source — and any assignment statement inside the formula writes to the host record before it is persisted.

Configure under Administration → Entity Manager → [Entity] → Formula. The formula box accepts one or more statements separated by semicolons. Examples of typical statements:

```
totalSessions = entity\countRelated('engagementSessions');

fullName = string\concatenate(firstName, ' ', lastName);

ifThen(
    status == 'Active' && closeDate == null,
```

```
    closeDate = datetime\today()
);
```

The formula has access to every field on the host record by name, plus the full library of formula functions. The entity\countRelated and entity\sumRelated functions reach across links to operate on related records — details in Section 15.6.

> **Note:** Before Save formulas fire on every save, including saves triggered by Workflow actions. If a workflow writes to the host record, the Before Save formula will re-run afterward. This is usually what you want (the count gets recalculated whenever anything saves the host), but it does mean expensive formulas can be invoked frequently. Keep them lean.

15.4 The Counted-Field Pattern: Workflow Plus Formula

A common requirement is to display a count of related records on the parent entity — for example, Total Sessions on an Engagement, showing how many Session records are linked. The pattern combines two mechanisms because neither one alone keeps the count accurate in every scenario.

A Before Save formula on the parent, written as totalSessions = entity\countRelated('engagementSessions'), computes the count correctly whenever the parent is saved. But adding a child record through the parent's bottom panel does not save the parent — only the child is saved. The count on the parent stays at its previous value until something causes the parent to save again.

A Workflow on the child entity, triggered After Record Created, can close that gap. The workflow's Update Related Record action reaches back through the child's link to the parent and recomputes the count using the same formula. Pair this with an After Record Removed workflow so deletes also decrement the count.

Keeping both the workflow and the Before Save formula in place is a deliberate belt-and-suspenders design:

- The workflow handles live updates as children are added and removed.
- The Before Save formula handles backfill of existing parent records (which never triggered a workflow because their children pre-date the workflow's installation) and any case where a child is re-linked to a different parent and the original parent's count needs refreshing on its next save.

The pattern is reusable: any time a parent entity needs to display a count, sum, or other aggregate of its related children, apply both layers. Section 15.5 walks through the specific Total Sessions configuration.

15.5 Step-by-Step: Total Sessions Count on Engagement

This walkthrough configures a Total Sessions field on the Engagement entity that displays the number of Session records linked to that Engagement. It assumes the Engagement-to-Session one-to-many relationship already exists (with link name engagementSessions on the Engagement side and engagement on the Session side); the relationship setup itself is covered in Section 2.

Step 1 — Add the Field on Engagement

Navigate to: Administration → Entity Manager → Engagement → Fields → Add Field. Configure the field as follows:

Field	Value	Notes
Type	Int	Whole number; the count will always be a non-negative integer.
Name	totalSessions	camelCase. This is the system name used in the formula.
Label	Total Sessions	Display label shown in the UI.
Read-only	Yes	Prevents users from overwriting the formula's output.
Default	(blank)	Leave empty. The formula and workflow set the value.

Click Save. Then add the field to the Engagement Detail View layout (Layouts → Detail View) so it is visible on the record page.

Step 2 — Add the Before Save Formula on Engagement

Navigate to: Administration → Entity Manager → Engagement → Formula. Add the following line to the formula box (preserving any existing formulas already present):

```
totalSessions = entity\countRelated('engagementSessions');
```

Save. Every subsequent Engagement save will recompute the count, including saves triggered by the workflow added in Step 3.

> **Note:** The link name passed to entity\countRelated is the link as it lives on the host entity (Engagement). EspoCRM does not always default this to the obvious name — in this case the link is engagementSessions, not sessions. To confirm the exact link name, open Administration → Entity Manager → Engagement → Relationships → [click the Sessions link] and read the Name field on the Sessions (right) side of the dialog. Section 15.7 covers this pitfall in more detail.

Step 3 — Create the After-Created Workflow on Session

This workflow keeps the count fresh when a new Session is created and linked to an Engagement. Navigate to: Administration → Workflows → Create Workflow. Configure as follows:

Field	Value
Name	TotalSessionCount (or any descriptive name)
Active	Checked
Target Entity Type	Session
Trigger Type	After record created

Conditions	(none)

Add one action of type Update Related Record. In the action configuration:

- Link — set to engagement (the link on Session that points back to its parent Engagement).
- Formula — leave the Add Field section alone and put the calculation in the Formula text box:

```
totalSessions = entity\countRelated('engagementSessions');
```

The formula runs in the context of the target record being updated (the Engagement reached through the engagement link), so entity\countRelated refers to that Engagement's sessions. Save the workflow.

Step 4 — Create the After-Removed Workflow on Session

Without a matching After Record Removed workflow, deleting a Session leaves the count one too high on the parent Engagement until the next save of that Engagement triggers the Before Save formula. Adding the removal workflow keeps the count instantly accurate.

Create a second workflow with the same configuration as Step 3, except:

- Trigger Type — After record removed.

Use the same Update Related Record action with the same Link and Formula as in Step 3. Save.

Step 5 — Verify and Backfill

To verify the configuration, create a new Session linked to any Engagement and refresh the Engagement's detail page. Total Sessions should now reflect the count. To backfill existing Engagements whose Sessions pre-date the workflows, open each Engagement and click Save without making any changes — the Before Save formula will populate the count. Bulk backfill can also be done through the API by issuing a no-op update to each Engagement record.

> **Note:** On a freshly added field, the value displays as None (not 0) on records where neither the workflow nor the Before Save formula has run yet. Once either mechanism fires, the field shows the computed value. If a record genuinely has zero related records, the field shows 0 — distinguishing "never computed" from "computed and zero" is useful when troubleshooting.

15.6 Common Formula Functions for Calculated Fields

The functions below are the ones most commonly needed for calculated-field work. The full reference is at the EspoCRM documentation; this table captures the subset that recurs across CBM's configuration.

Function	What it does
entity\countRelated(LINK)	Returns the number of records related through the given link on the host entity. LINK is the link name on the host side, not the foreign entity's type.
entity\countRelated(LINK, FILTER_KEY, VALUE)	Counts related records that match a simple field-value filter — e.g., entity\countRelated('engagementSessions', 'status=',

	'Held') counts only held sessions.
entity\sumRelated(LINK, FIELD)	Returns the sum of FIELD across all records related through LINK. Useful for total-amount roll-ups.
entity\sumRelated(LINK, FIELD, FILTER_KEY, VALUE)	Same as sumRelated but with a filter applied to the related records before summing.
entity\isNew()	Returns true if the host record is being created (vs updated). Use this to guard logic that should only run once at creation — e.g., setting a default value if a field was left blank.
entity\isAttributeChanged('field')	Returns true if the named field changed in this save. Useful for triggering computations only when a specific field is touched.
record\count('Entity', KEY, VALUE)	Counts records of any entity type matching a filter — independent of any relationship to the host. Use when you need a count that is not scoped through a link.
string\concatenate(...)	Concatenates a list of strings into one. Common for assembling display names or composite identifiers.
ifThen(CONDITION, STATEMENT)	Executes STATEMENT only if CONDITION is true. The conditional flow primitive.
datetime\today()	Returns today’s date as a date value. datetime\now() returns the current timestamp.
env\userAttribute('id')	Returns an attribute of the current user (the user who triggered the save). Common attributes: id, name, emailAddress, teamsIds.

Filter strings in countRelated and sumRelated use the same operators as record\count: = (equals), != (not equals), >, <, >=, <=, * (LIKE), and !* (NOT LIKE). For more complex filters that cannot be expressed as a single key-value pair, define a Report Filter at Administration → Report Filters and reference it by ID as reportFilter{filterId} in the formula — see the EspoCRM documentation for details. Report Filters require the Advanced Pack.

15.7 Troubleshooting and Gotchas

The most common failure modes when setting up calculated fields:

- Link name does not match the obvious default. EspoCRM sometimes prepends the parent entity name when generating a link name — the Sessions link on Engagement is engagementSessions, not sessions. The formula will silently return 0 (or throw a server-side error logged but not surfaced) if the link name is wrong. Confirm the exact link name in Administration → Entity Manager → [Entity] → Relationships before writing the formula.
- Trigger type mismatch when testing. After Record Created fires only on the initial create event. Editing an existing record does not trigger this workflow. When verifying a new workflow, always test by creating a fresh child record, not by editing an existing one. If a single trigger needs to cover both create and update, use After Record Saved.
- None vs 0 on the calculated field. A display value of None means neither the workflow nor the Before Save formula has ever written to that field on this record. A display value of 0 means the formula did run and the actual count is zero. If you expected a non-zero count and see None, the formula has not fired — save the record manually to verify the Before Save formula works, then dig into why the workflow did not.

- Workflow log shows the workflow ran, but the field did not update. The Update Related Record action will silently no-op if the Link points to a record that does not exist (the child was not properly linked to a parent at the moment the workflow fired). For After Record Created workflows on a child with a required parent link, this is usually fine; for entities where the parent link is optional, check that the child is actually linked at creation time.
- Backfill is not automatic. Workflows fire only on events that happen after the workflow is created. Existing parent records with existing children will show None until the parent is saved (triggering the Before Save formula) or until the count is set manually through the API or an import. Plan for backfill when introducing a new calculated field on an existing data set.
- Re-linking children does not always refresh the old parent. If a child is moved from Engagement A to Engagement B, the After-Created workflow runs against B (correct) but not against A. A's count is now stale until A is saved. The Before Save formula on the parent mitigates this on the next save of A; for always-current accuracy, add an After Record Updated workflow with a condition on the parent link field changing.

Questions for Research

The following questions arose during configuration and require further research before they can be documented as How To steps.

- How to add a filtered view to the left navigation menu alongside other entities — for example, adding a 'My Engagements' or 'Client Companies' entry to the left column that opens a pre-filtered list view. No admin UI path has been identified for this. It likely requires custom development such as a Custom View or metadata configuration, but the exact approach needs to be researched and documented.
- Whether EspoCRM supports multiple list views with different column layouts for different types within the same entity — for example, showing different columns for Client Companies vs Partner Companies. Saved search filters can show different subsets of records but cannot change the column layout per type. EspoCRM does support Custom Views (a developer-level JavaScript customization) that could dynamically show different columns based on a field value, but this requires writing and maintaining custom JS files on the server — similar in complexity to the Dynamic Handler approach. This is not a practical solution for most admin use cases and should be considered only if the difference in fields between types is significant enough to justify the development effort.
- How to prevent users from using the Remove function in relationship panels — Remove permanently deletes the linked record entirely. Regular users should not have this ability. The exact Role permission that controls this needs to be identified and documented.
- How to prevent users from using the Unlink function in relationship panels — Unlink removes the association between two records without deleting either record. Whether this should be restricted for regular users, and the Role permission that controls it, needs to be identified and documented.
- How to show all emails sent to a contact on their Contact detail page — there is no Emails panel available in the Contact entity's Bottom Panels layout, suggesting email display is handled through a different mechanism such as email archiving or the activity

stream. The correct configuration path needs to be identified and documented.

- How do users add notes to a record — the Note entity exists in Entity Manager as a system-level entity with limited configurability (Fields only, no Layouts or Relationships tabs). It likely powers an activity stream or notes panel on record detail pages, but the exact user-facing path for posting notes, attaching files, and viewing note history on a record needs to be identified and documented.

Change Log

Version	Date	Author	Description
2.6	06-12-26 20:51	Doug Bower	Added new top-level heading “Dashboards and Dashlets” between Lists and Filter Configuration and Security Configuration, with Section 16 of the same name. Covers dashboard concepts (per-user model, tabs, lock, admin reset), the dashlet types EspoCRM ships, the create-a-dashlet flow from the home view, Record List and Report dashlet configuration in detail, Dashboard Templates and team-based distribution, and a worked “My Active Engagements” example using a Report Complex Expression with “env\userAttribute('id')” for current-user scoping. Removed three resolved items from the Questions for Research section: relationship-panel filter exposure (now in 8.5), system-level saved searches (now in 7.5), and current-user filtering (now in 8.4 and 16.7).
2.5	06-11-26 16:30	Doug Bower	Added Section 8 “Front-End Configuration with clientDefs” under the Lists and Filter Configuration heading. Covers the clientDefs metadata type and its boundary with selectDefs, file location and recursive merge, the parameter landscape, exposing the built-in Only My / bool filters and default filters, relationship-panel select filtering, a custom filter spanning clientDefs + selectDefs, dynamic logic, and the mapping to CRM Builder YAML.
2.4	06-11-26 15:56	Doug Bower	Added Section 7 “Reports and Report-Based Filters” under the Lists and Filter Configuration heading. Covers the three-way terminology disambiguation (record filters vs. runtime filters vs. list-view Report Filters), the seven record-filter constructs (Field, OR/AND groups, NOT IN/IN groups, Complex Expression, Having group), the three runtime-filter usage modes (interactive, dashlet, report-panel auto-fill),

			an expanded Complex Expression subsection (syntax rules, the Formula-side limitations, full function reference, four CBM-shaped worked examples, operator gotchas), and the Administration > Report Filters list-view feature with its formula-function reuse pattern.
2.3	06-02-26 19:15	Doug Bower	Added Section 10 "Message Templates and the Template Manager" under the Email Configuration heading. Covers the Template Manager scope (system-generated messages, fixed list), editing system templates with {{fieldName}} placeholders, the distinction from workflow and manual Email Templates and the silent blank-render gotcha, and the Foreign-field workaround for reaching related-entity data in invitations and notifications.
2.2	05-22-26 18:47	Doug Bower	Added new top-level Automation Configuration heading with Section 15 "Formulas and Calculated Fields". Covers the formula vs. workflow decision framework, Before Save formulas on an entity, the counted-field pattern (workflow plus formula), a worked Total Sessions on Engagement example, a reference of common formula functions, and troubleshooting guidance for link-name mismatches, trigger-type pitfalls, and backfill considerations.
2.1	05-20-26 03:48	Doug Bower	Added Section 14 "User Auto-Assignment" under the Security Configuration heading. Documents the three-layer behavior (default pre-fill, user preference, Before Save formula), the Multiple Assigned Users variant, Default Team Set, and CBM-specific application guidance.
2.0	05-20-26 05:45	Doug Bower	Document rebuilt from May 11 backup after file corruption. Added Sections 12 (EspoCRM Security Concepts) and 13 (CBM Security Implementation) under the existing Security Configuration heading. Saved as proper binary .docx to prevent recurrence.
1.0	05-05-26 11:03	Doug Bower	Initial baseline — existing content prior to revision control adoption.
1.1	05-05-26 11:03	Doug Bower	Added Section 1.2 "Entity Types"; renumbered subsequent subsections in Section 1; updated Type-field note in the entity-creation procedure.

1.2	05-10-26 04:27	Doug Bower	Added Section 2.8 "Foreign Fields — Displaying Linked Entity Data" covering concept, configuration steps, an Engagement-to-Mentor example, and operational constraints.
1.3	05-11-26 19:21	Doug Bower	Added Section 2.9 "One-to-One Relationships — In Depth" covering definition, the One-to-One Right vs One-to-One Left decision, what EspoCRM creates under the hood, uniqueness enforcement, step-by-step creation, a Contact-to-Mailing-Platform-Profile example, foreign fields across the link, and pitfalls including when NOT to use one-to-one.